十一届省赛第一套

```c
#include <STC15F2K60S2.H>
#include "stdio.h"
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "iic.h"
//按键
idata unsigned char key_val,key_old,key_down,key_up;
idata unsigned char key_slow_down;

//数码管
idata unsigned char seg_pos;
pdata unsigned char seg_buf[]={10,10,10,10,10,10,10,10};
idata unsigned char seg_slow_down;
idata unsigned char seg_disp_mode=0;//0-数据界面 1-参数设置界面
2-计数界面
//led
pdata unsigned char ucled[]={0,0,0,0,0,0,0,0};
idata unsigned int time_5s;//led 5s计时器
idata bit L1_flag;//当前电压小于电压参数标志位 1-小于
//电压值
idata unsigned char ad_slow_down;
idata unsigned int V_100x;//电压值100倍
idata unsigned int V_old_100x;//上一次的电压值100倍
idata unsigned char V_para_10x;//电压参数值10倍 0-50
//计数
idata unsigned int count=0;
idata unsigned int error_key_count;//无效按键操作次数
void key_proc()
{
```

```
    if(key_slow_down<10) return;
    key_slow_down=0;
    key_val=key_read();
    key_down=key_val&(key_val^key_old);
    key_up=~key_val&(key_val^key_old);
    key_old=key_val;
    if(key_down==12)
    {
        switch(seg_disp_mode)
        {
            case 0:
                seg_disp_mode=1;
                break;
            case 1:
                eeprom_write(&V_para_10x,0,1);
                seg_disp_mode=2;
                break;
            case 2:
                seg_disp_mode=0;
                break;
        }
    }

    if(key_down==13)
    {
        if(seg_disp_mode==2)
        {
            error_key_count=0;
            count=0;
        }
        else
            error_key_count++;
    }

        if(key_down==16)
        {
```

```
            if(seg_disp_mode==1)
            {
                error_key_count=0;
                if(v_para_10x>=50) //如果已经达到5v，那么再按下按键，回到0v
                    v_para_10x=0;
                else //否则+0.5v
                    v_para_10x+=5;
            }
            else
                error_key_count++;
        }

        if(key_down==17)
        {
            if(seg_disp_mode==1)
            {
                error_key_count=0;
                if(v_para_10x==0) //如果按下按键前已经达到0v，那么按下按键，回到5v
                    v_para_10x=50;
                else
                    v_para_10x-=5;
            }
            else
                error_key_count++;
        }
}

void seg_proc()
{
    if(seg_slow_down<80) return;
    seg_slow_down=0;

    switch(seg_disp_mode)
    {
```

```
            case 0: //数据界面
                seg_buf[0]=11;
                seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=10;
                seg_buf[5]=v_100x/100%10+',';
                seg_buf[6]=v_100x/10%10;
                seg_buf[7]=v_100x %10;
                break;
            case 1: //参数设置界面
                seg_buf[0]=12;
                seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=10;
                seg_buf[5]=v_para_10x/10%10+',';
                seg_buf[6]=v_para_10x%10;
                seg_buf[7]=0;
                break;
            case 2: //计数界面
                seg_buf[0]=13;
                seg_buf[1]=(count>=1000000)?
count/1000000%10:10;
                seg_buf[2]=(count>=100000)?count/100000%10:10;
                seg_buf[3]=(count>=10000)?count/10000%10:10;
                seg_buf[4]=(count>=1000)?count/1000%10:10;
                seg_buf[5]=(count>=100)?count/100%10:10;;
                seg_buf[6]=(count>=10)?count/10%10:10;
                seg_buf[7]=count %10;
                break;
        }
}

void led_proc()
{
    if(v_100x<v_para_10x*10&&time_5s>5000)
    {
        L1_flag=1;
    }
    else
    {
```

```
            L1_flag=0;
        }
        ucled[0]=L1_flag;
        ucled[1]=(count%2!=0);
        ucled[2]=(error_key_count>=3);
    }

    void get_ad()
    {
        if(ad_slow_down<100) return;
        ad_slow_down=0;
        V_old_100x=V_100x;
        ad_read(0x03);
        V_100x=(ad_read(0x03)/51.0)*100;
        if(V_old_100x>V_para_10x*10 && V_100x<V_para_10x*10)
        {
            count++;
        }

    }
    void Timer1_Init(void)      //1毫秒@12.000MHz
    {
        AUXR &= 0xBF;           //定时器时钟12T模式
        TMOD &= 0x0F;           //设置定时器模式
        TL1 = 0x18;             //设置定时初始值
        TH1 = 0xFC;             //设置定时初始值
        TF1 = 0;                //清除TF1标志
        TR1 = 1;                //定时器1开始计时
        ET1 = 1;                //使能定时器1中断
        EA  = 1;          //使能全局中断
    }

    void Timer1_Isr(void) interrupt 3
    {
        key_slow_down++;
        seg_slow_down++;
```

```
        ad_slow_down++;
        if(seg_buf[seg_pos]>20)
            seg_disp(seg_pos,seg_buf[seg_pos]-',',1);
        else
            seg_disp(seg_pos,seg_buf[seg_pos],0);
        seg_pos=(++seg_pos)%8;
        if(V_100x<V_para_10x*10)
        {
            if(++time_5s==5000) time_5s=5001;
        }
        else
        {
            time_5s=0;
        }

        led_disp(ucled);
    }
    void main()
    {
        system_init();
        Timer1_Init();
        eeprom_read(&V_para_10x,0,1);
        while(1)
        {
            key_proc();
            seg_proc();
            led_proc();
            get_ad();
        }
    }
```

十二届省赛第一套

```c
#include <STC15F2K60S2.H>
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "onewire.h"
#include "iic.h"

//按键
idata unsigned char key_val,key_old,key_down,key_up;
idata unsigned char key_slow_down;

//数码管
idata unsigned char seg_pos;
pdata unsigned char seg_buf[]={10,10,10,10,10,10,10,10};
idata unsigned char seg_slow_down;
idata unsigned char seg_disp_mode=0;//0-温度显示界面 1-参数设置界面 2-DAC输出界面
//led
pdata unsigned char ucled[]={0,0,0,0,0,0,0,0};

//温度
idata unsigned int T_val_100x;//温度数据100倍
idata unsigned char T_para=25;//温度参数
idata unsigned char T_para_set;//温度参数设置变量
idata unsigned int temp_slow_down;
idata bit got_T_flag;
//DAC
idata unsigned int V_val_100x;//DAC输出电压值的100倍
idata bit model_flag=0;//模式标志位 0-模式1 1-模式2
void key_proc()
```

```
{
    if(key_slow_down<10) return;
    key_slow_down=0;
    key_val=key_read();
    key_down=key_val&(key_val^key_old);
    key_up=~key_val&(key_val^key_old);
    key_old=key_val;
    if(key_down==4)
    {
        switch(seg_disp_mode)
        {
            case 0:
                T_para_set=T_para;//进入设置界面前把参数值赋值给参数设置值
                seg_disp_mode=1;
                break;
            case 1:
                T_para=T_para_set;//离开参数设置界面前，把设置值赋值给参数值
                seg_disp_mode=2;
                break;
            case 2:
                seg_disp_mode=0;
                break;
        }
    }
    if(seg_disp_mode==1)
    {
        if(key_down==8)
        {
            if(T_para_set>0)
                T_para_set--;
        }
        if(key_down==9)
        {
            if(T_para_set<99)
```

```
                T_para_set++;
            }
        }
        if(key_down==5)
            model_flag=!model_flag;
    }


    void seg_proc()
    {
        if(seg_slow_down<80) return;
        seg_slow_down=0;

        switch(seg_disp_mode)
        {
            case 0:
                seg_buf[0]=11;
                seg_buf[1]=seg_buf[2]=seg_buf[3]=10;
                seg_buf[4]=T_val_100x/1000%10;
                seg_buf[5]=T_val_100x/100%10+',';
                seg_buf[6]=T_val_100x/10%10;
                seg_buf[7]=T_val_100x%10;
                break;
            case 1:
                seg_buf[0]=12;

    seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=seg_buf[5]=10;
                seg_buf[6]=T_para_set/10%10;
                seg_buf[7]=T_para_set%10;
                break;
            case 2:
                seg_buf[0]=13;
                seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=10;
                seg_buf[5]=V_val_100x/100%10+',';
                seg_buf[6]=V_val_100x/10%10;
                seg_buf[7]=V_val_100x%10;
```

```
            break;
        }
    }


    void led_proc()
    {
        ucled[0]=(!model_flag);
        ucled[1]=(seg_disp_mode==0);
        ucled[2]=(seg_disp_mode==1);
        ucled[3]=(seg_disp_mode==2);
    }


    void get_temperature()
    {
        float temp;
        if(temp_slow_down<300) return;
        temp_slow_down=0;
        temp=rd_temperature();
        T_val_100x=(unsigned int)(temp*100);
        got_T_flag=1;
    }

    void da_proc()
    {
        if(got_T_flag)
        {
            got_T_flag=0;
            if(model_flag==0)
            {
                if(T_val_100x<T_para*100)
                    V_val_100x=0;
                else
                    V_val_100x=500;
            }
```

```
        else
        {
            if(T_val_100x>=4000)
                V_val_100x=400;
            else if(T_val_100x>2000)
                V_val_100x=(unsigned int)((((3.0/20.0)*
(T_val_100x/100.0-20))+1.0)*100);
            else
                V_val_100x=100;
        }
        da_write((float)V_val_100x/100.0*51.0);
    }
}
void Timer1_Init(void)       //1毫秒@12.000MHz
{
    AUXR &= 0xBF;            //定时器时钟12T模式
    TMOD &= 0x0F;            //设置定时器模式
    TL1 = 0x18;              //设置定时初始值
    TH1 = 0xFC;              //设置定时初始值
    TF1 = 0;                 //清除TF1标志
    TR1 = 1;                 //定时器1开始计时
    ET1 = 1;                 //使能定时器1中断
    EA  = 1;         //使能全局中断
}

void Timer1_Isr(void) interrupt 3
{
    key_slow_down++;
    seg_slow_down++;
    temp_slow_down++;
    if(seg_buf[seg_pos]>20)
        seg_disp(seg_pos,seg_buf[seg_pos]-',',1);
    else
        seg_disp(seg_pos,seg_buf[seg_pos],0);
    seg_pos=(++seg_pos)%8;
```

```
    led_disp(ucled);
}


void main()
{
    system_init();
    Timer1_Init();
    while(1)
    {
        key_proc();
        seg_proc();
        led_proc();
        get_temperature();
        da_proc();
    }
}
```

Fence 2

十三届国赛

```c
#include <STC15F2K60S2.H>
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "iic.h"
#include "ultrasound.h"
//频率
idata unsigned int freq;
idata unsigned int freq_time_1s;
idata unsigned char freq_para_10x=90;//(10-120)1.0-12.0 单位kHz
//按键
idata unsigned char key_val,key_old,key_down,key_up;
idata unsigned char key_slow_down;
idata bit key_long_flag;
idata unsigned int key_long_tick;
//数码管
idata unsigned char seg_pos;
pdata unsigned char seg_buf[]={10,10,10,10,10,10,10,10};
idata unsigned char seg_slow_down;
idata unsigned char seg_disp_mode=0;//0-频率界面 1-湿度界面 2-测距界面 3-参数界面
idata bit freq_danwei=0;//0-Hz 1-kHz
idata bit dist_danwei=0;//0-cm 1-m
idata unsigned char para_disp_mode;//0-频率参数 1-湿度参数 2-距离参数
//led
pdata unsigned char ucled[]={0,0,0,0,0,0,0,0};
idata bit L1_flash_flag;
idata bit L2_flash_flag;
```

```
idata bit L3_flash_flag;
idata unsigned char led1_flash_tick;
idata unsigned char led2_flash_tick;
idata unsigned char led3_flash_tick;
//继电器
idata unsigned char relay_count;
idata bit relay_old = 0;
idata bit relay_count_dirty = 0;
//电机
idata unsigned char motor_period;//0-9
idata unsigned char motor_compare;
//ad
idata unsigned char ad_slow_down;
//超声波
idata unsigned char dist_slow_down;
idata unsigned char dist_val;
idata unsigned char dist_para_10x=6;//单位m (0.1-1.2)1-12
//湿度
idata unsigned char w_val;
idata unsigned char w_para=40;//10-60 单位：百分之
//eeprom
idata unsigned char relay_count_pre = 0;
idata unsigned char eeprom_lock = 13;
idata unsigned char eeprom_temp = 0;
void key_proc()
{
    if(key_slow_down<10) return;
    key_slow_down=0;
    key_val=key_read();
    key_down=key_val&(key_val^key_old);
    key_up=~key_val&(key_val^key_old);
    key_old=key_val;
    if(key_down==4)
    {
        seg_disp_mode=(++seg_disp_mode)%4;
        para_disp_mode=0;
```

```
            led_off();
        }
        if(seg_disp_mode==3)
        {
            if(key_down==5)
            {
                para_disp_mode=(++para_disp_mode)%3;
            }
            if(key_down==6)
            {
                if(para_disp_mode==0)
                {
                    if(freq_para_10x>=120)
                        freq_para_10x=10;
                    else
                        freq_para_10x+=5;
                }
                else if(para_disp_mode==1)
                {
                    if(W_para>=60)
                        W_para=10;
                    else
                        W_para+=10;
                }
                else
                {
                    if(dist_para_10x>=12)
                        dist_para_10x=1;
                    else
                        dist_para_10x++;
                }
            }

            if(key_down==7)
            {
                if(para_disp_mode==0)
```

```
            {
                if(freq_para_10x<=10)
                    freq_para_10x=120;
                else
                    freq_para_10x-=5;
            }
            else if(para_disp_mode==1)
            {
                if(W_para<=10)
                    W_para=60;
                else
                    W_para-=10;
            }
            else
            {
                if(dist_para_10x<=1)
                    dist_para_10x=12;
                else
                    dist_para_10x--;
            }
        }
    }
    if(seg_disp_mode==2)
    {
        if(key_down==6)
            dist_danwei=!dist_danwei;
    }
    if(seg_disp_mode==0)
    {
        if(key_down==7)
            freq_danwei=!freq_danwei;
    }
    if(seg_disp_mode==1)
    {
        if(key_down==7)
        {
```

```
            key_long_flag=1;
        }
        if(key_up==7)
        {
            if(key_long_tick>=1000)
            {
                relay_count=0;
                relay_count_dirty = 1;
            }
            key_long_flag=0;
            key_long_tick=0;
        }
    }
}


void seg_proc()
{
    if(seg_slow_down<80) return;
    seg_slow_down=0;
    switch(seg_disp_mode)
    {
        case 0:
            seg_buf[0]=11;
            seg_buf[1]=10;
            if(freq_danwei==0)
            {
                seg_buf[2]=(freq>=100000)?
freq/100000%10:10;
                seg_buf[3]=(freq>=10000)?freq/10000%10:10;
                seg_buf[4]=(freq>=1000)?freq/1000%10:10;
                seg_buf[5]=(freq>=100)?freq/100%10:10;
                seg_buf[6]=(freq>=10)?freq/10%10:10;
                seg_buf[7]=freq%10;
            }
            else
```

```
            {
                seg_buf[2]=10;
                seg_buf[3]=10;
                seg_buf[4]=(freq>=100000)?
freq/100000%10:10;
                seg_buf[5]=(freq>=10000)?freq/10000%10:10;
                seg_buf[6]=freq/1000%10+',';
                seg_buf[7]=freq/100%10;
            }
            break;
        case 1:
            seg_buf[0]=12;

seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=seg_buf[5]=10;
            seg_buf[6]=W_val/10%10;
            seg_buf[7]=W_val%10;
            break;
        case 2:
            seg_buf[0]=13;
            seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=10;
            if(dist_danwei==0)
            {
                seg_buf[5]=(dist_val>=100)?
dist_val/100%10:10;
                seg_buf[6]=(dist_val>=10)?
dist_val/10%10:10;
                seg_buf[7]=dist_val%10;
            }
            else
            {
                seg_buf[5]=dist_val/100%10+',';
                seg_buf[6]=dist_val/10%10;
                seg_buf[7]=dist_val%10;
            }
            break;
        case 3:
```

```
            seg_buf[0]=14;
            if(para_disp_mode==0)
            {
                seg_buf[1]=1;
                seg_buf[2]=seg_buf[3]=seg_buf[4]=10;
                seg_buf[5]=(freq_para_10x>=100)?
freq_para_10x/100%10:10;
                seg_buf[6]=freq_para_10x/10%10+',';
                seg_buf[7]=freq_para_10x%10;
            }
            else if(para_disp_mode==1)
            {
                seg_buf[1]=2;

seg_buf[2]=seg_buf[3]=seg_buf[4]=seg_buf[5]=10;
                seg_buf[6]=W_para/10%10;
                seg_buf[7]=W_para%10;
            }
            else
            {
                seg_buf[1]=3;

seg_buf[2]=seg_buf[3]=seg_buf[4]=seg_buf[5]=10;
                seg_buf[6]=dist_para_10x/10%10+',';
                seg_buf[7]=dist_para_10x%10;
            }
            break;

    }
}

void led_proc()
{
    switch(seg_disp_mode) //0-频率界面 1-湿度界面 2-测距界面 3-参
数界面
    {
```

```
            case 0:
                ucled[0]=1;
                ucled[1]=0;
                ucled[2]=0;
                break;
            case 1:
                ucled[0]=0;
                ucled[1]=1;
                ucled[2]=0;
                break;
            case 2:
                ucled[0]=0;
                ucled[1]=0;
                ucled[2]=1;
                break;
            case 3: //0-频率参数 1-湿度参数 2-距离参数
                if(para_disp_mode==0)
                {
                    ucled[0]=L1_flash_flag;
                    ucled[1]=0;
                    ucled[2]=0;
                }
                else if(para_disp_mode==1)
                {
                    ucled[1]=L2_flash_flag;
                    ucled[0]=ucled[2]=0;
                }
                else
                {
                    ucled[0]=ucled[1]=0;
                    ucled[2]=L3_flash_flag;
                }
                break;
        }
        ucled[3]=(freq>freq_para_10x*100);
        ucled[4]=(W_val>W_para);
```

```
    ucled[5]=(dist_val>dist_para_10x*10);
}



void relay_proc()
{
    bit relay_new = (dist_val > dist_para_10x * 10);

    if(relay_new != relay_old)
    {
            relay(relay_new);
            relay_old = relay_new;
            relay_count++;
            relay_count_dirty = 1;
    }
}

void eeprom_proc()
{
    if(!relay_count_dirty) return;
    eeprom_write(&relay_count, 0, 1);
    relay_count_dirty = 0;
}

void motor_proc()
{
    if(freq>freq_para_10x*100)
        motor_compare=8;//占空比为8
    else
        motor_compare=2;//占空比为2
}
void ad_proc()
{
    float temp1;
    float temp2;
```

```
    if(ad_slow_down<100) return;
    ad_slow_down=0;
    ad_read(0x43);
    temp1=ad_read(0x43)/51.0;
    if(temp1>5)
        W_val=100;
    else
        W_val=(unsigned char)(20*temp1);

    if(W_val>80)
        da_write(5*51);
    else if(W_val>=W_para)
    {
        temp2=(4.0/(float)(80-W_para))*(W_val-W_para)+1.0;
        da_write(temp2*51);
    }
    else
    {
        da_write(1*51);
    }
}


void dist_proc()
{
    if(dist_slow_down<100) return;
    dist_slow_down=0;
    dist_val=ut_wave_data();
}

void Timer0_Init(void)        //1毫秒@12.000MHz
{
    AUXR &= 0x7F;             //定时器时钟12T模式
    TMOD &= 0xF0;             //设置定时器模式
    TMOD |= 0x05;             //设置定时器模式 为P34引脚读取外部脉冲
模式
```

```
    TL0 = 0x00;                 //设置定时初始值
    TH0 = 0x00;                 //设置定时初始值
    TF0 = 0;                    //清除TF0标志
    TR0 = 1;                    //定时器0开始计时
}



void Timer1_Init(void)      //100微秒@12.000MHz
{
    AUXR &= 0xBF;               //定时器时钟12T模式
    TMOD &= 0x0F;               //设置定时器模式
    TL1 = 0x9C;                 //设置定时初始值
    TH1 = 0xFF;                 //设置定时初始值
    TF1 = 0;                    //清除TF1标志
    TR1 = 1;                    //定时器1开始计时
    ET1 = 1;                    //使能定时器1中断
    EA = 1;             //使能全局中断
}



void Timer2_Init(void)      //1毫秒@12.000MHz
{
    AUXR &= 0xFB;               //定时器时钟12T模式
    T2L = 0x18;                 //设置定时初始值
    T2H = 0xFC;                 //设置定时初始值
    AUXR |= 0x10;               //定时器2开始计时
    IE2 |= 0x04;                //使能定时器2中断
    EA = 1;               //使能全局中断
}

void Timer1_Isr(void) interrupt 3
{
    if(motor_period<motor_compare)
        motor(1);
```

```
    else
        motor(0);
    motor_period=(++motor_period)%10;
}
void Timer2_Isr(void) interrupt 12
{
    key_slow_down++;
    seg_slow_down++;
    ad_slow_down++;
    dist_slow_down++;
    if(seg_buf[seg_pos]>20)
        seg_disp(seg_pos,seg_buf[seg_pos]-',',1);
    else
        seg_disp(seg_pos,seg_buf[seg_pos],0);
    seg_pos=(++seg_pos)%8;
    if(++freq_time_1s==1000)
    {
        freq_time_1s=0;
        freq=TH0<<8|TL0;
        TH0=TL0=0;
    }
    if(key_long_flag)
    {
        if(++key_long_tick>=1000) key_long_tick=1001;
    }
    else
    {
        key_long_tick=0;
    }

    if(seg_disp_mode==3&&para_disp_mode==0) //0-频率参数 1-湿
度参数 2-距离参数
    {
        if(++led1_flash_tick==100)
        {
            led1_flash_tick=0;
```

```
            L1_flash_flag=!L1_flash_flag;
        }
    }
    else
    {
        led1_flash_tick=0;
        L1_flash_flag=0;
    }

    if(seg_disp_mode==3&&para_disp_mode==1) //0-频率参数 1-湿度参数 2-距离参数
    {
        if(++led2_flash_tick==100)
        {
            led2_flash_tick=0;
            L2_flash_flag=!L2_flash_flag;
        }
    }
    else
    {
        led2_flash_tick=0;
        L2_flash_flag=0;
    }

    if(seg_disp_mode==3&&para_disp_mode==2) //0-频率参数 1-湿度参数 2-距离参数
    {
        if(++led3_flash_tick==100)
        {
            led3_flash_tick=0;
            L3_flash_flag=!L3_flash_flag;
        }
    }
    else
    {
        led3_flash_tick=0;
```

```
            L3_flash_flag=0;
        }
        led_disp(ucled);
    }

    void main()
    {
        system_init();
        eeprom_read(&relay_count, 0, 1);
        Timer0_Init();
        Timer1_Init();
        Timer2_Init();
        while(1)
        {
            key_proc();
            seg_proc();
            led_proc();
            dist_proc();
            relay_proc();
            eeprom_proc();
            motor_proc();
            ad_proc();
        }

    }
```

十三届省赛第一套

```c
#include <STC15F2K60S2.H>
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "onewire.h"
#include "ds1302.h"
//按键
idata unsigned char key_val,key_old,key_down,key_up;
idata unsigned char key_slow_down;
idata bit key_long_flag;//按键17长按标志位
//数码管
idata unsigned char seg_pos;
pdata unsigned char seg_buf[]={10,10,10,10,10,10,10,10};
idata unsigned char seg_slow_down;
idata unsigned char seg_disp_mode=0;//0-温度显示界面 1-时间显示界面 2-参数设置界面

//led
pdata unsigned char ucled[]={0,0,0,0,0,0,0,0};
idata unsigned char led_time_100ms;//led闪烁计时器
idata bit led_flash_flag;//led闪烁标志位
idata bit relay_flag;//继电器吸合标志位
//温度
idata unsigned int T_val_10x;//温度10倍
idata unsigned char T_para=23;//10-99
idata unsigned int temp_slow_down;
//时间
pdata unsigned char ucrtc[]={12,59,55};
//模式控制
idata bit model_flag=0;//0-温度控制模式 1-时间控制模式
```

```
idata unsigned int time_5s;//继电器，led5s计时器
idata bit time_flag;//整点标志位

void key_proc()
{
    if(key_slow_down<10) return;
    key_slow_down=0;
    key_val=key_read();
    key_down=key_val&(key_val^key_old);
    key_up=~key_val&(key_val^key_old);
    key_old=key_val;
    if(key_down==12)
        seg_disp_mode=(++seg_disp_mode)%3;//0,1,2之间循环
    if(key_down==13)
        model_flag=!model_flag;
    if(seg_disp_mode==2)
    {
        if(key_down==16)
        {
            if(T_para<99) T_para++;
        }
        if(key_down==17)
        {
            if(T_para>10) T_para--;
        }
    }
    if(seg_disp_mode==1)
    {
        if(key_old==17)
        {
            key_long_flag=1;
        }
        if(key_up==17)
        {
            key_long_flag=0;
        }
```

```
    }
}

void seg_proc()
{
    if(seg_slow_down<80) return;
    seg_slow_down=0;
    read_rtc(ucrtc);

    switch(seg_disp_mode)
    {
        case 0:
            seg_buf[0]=11;
            seg_buf[1]=1;
            seg_buf[2]=seg_buf[3]=seg_buf[4]=10;
            seg_buf[5]=T_val_10x/100%10;
            seg_buf[6]=T_val_10x/10%10+',';
            seg_buf[7]=T_val_10x%10;
            break;
        case 1:
            seg_buf[0]=11;
            seg_buf[1]=2;
            seg_buf[2]=10;
            seg_buf[5]=12;
            if(key_long_flag==0)
            {
                seg_buf[3]=ucrtc[0]/10%10;
                seg_buf[4]=ucrtc[0]%10;
                seg_buf[6]=ucrtc[1]/10%10;
                seg_buf[7]=ucrtc[1]%10;
            }
            else
            {
                seg_buf[3]=ucrtc[1]/10%10;
                seg_buf[4]=ucrtc[1]%10;
                seg_buf[6]=ucrtc[2]/10%10;
```

```
                seg_buf[7]=ucrtc[2]%10;
            }
            break;
        case 2:
            seg_buf[0]=11;
            seg_buf[1]=3;
            seg_buf[2]=seg_buf[3]=seg_buf[4]=seg_buf[5]=10;
            seg_buf[6]=T_para/10%10;
            seg_buf[7]=T_para%10;
            break;
    }
}

void led_relay_proc()
{
    if(ucrtc[1]==0&&ucrtc[2]==0)//整点时，标志位置为1
    {
        time_flag=1;
        time_5s=0;  //初始化5s计时器
    }
    if(model_flag==0)//温度控制模式
    {
        if(T_val_10x>(T_para*10))
        {
            relay(1);//开发板上的L10指示灯点亮，就表示继电器吸合
            relay_flag=1;
        }
        else
        {
            relay(0);
            relay_flag = 0;
        }
    }
    else //时间控制模式
    {
            if(time_flag)
```

```
            {
                relay(1);
                relay_flag=1;
            }
            else
            {
                relay(0);
                relay_flag = 0;   //继电器断开，就清零
            }
    }

    ucled[0]=time_flag;
    ucled[1]=(!model_flag);
    ucled[2]=led_flash_flag;
}

void get_temperature()
{
    idata float temp;
    if(temp_slow_down<300) return;
    temp_slow_down=0;
    temp=rd_temperature();
    T_val_10x=(unsigned int)(temp*10);
}

void Timer1_Init(void)       //1毫秒@12.000MHz
{
    AUXR &= 0xBF;            //定时器时钟12T模式
    TMOD &= 0x0F;            //设置定时器模式
    TL1 = 0x18;              //设置定时初始值
    TH1 = 0xFC;              //设置定时初始值
    TF1 = 0;                 //清除TF1标志
    TR1 = 1;                 //定时器1开始计时
    ET1 = 1;                 //使能定时器1中断
    EA = 1;          //使能全局中断
}
```

```
void Timer1_Isr(void) interrupt 3
{
    key_slow_down++;
    seg_slow_down++;
    temp_slow_down++;
    if(seg_buf[seg_pos]>20)
        seg_disp(seg_pos,seg_buf[seg_pos]-',',1);
    else
        seg_disp(seg_pos,seg_buf[seg_pos],0);
    seg_pos=(++seg_pos)%8;

    if(time_flag)
    {
        if(++time_5s==5000)
        {
            time_5s=0;
            time_flag=0;
        }
    }

    if(relay_flag)
    {
        if(++led_time_100ms==100)
        {
            led_time_100ms=0;
            led_flash_flag=!led_flash_flag;
        }
    }
    else
    {
        led_time_100ms=0;
        led_flash_flag=0;
    }
    led_disp(ucled);
}
```

```
void main()
{
    system_init();
    set_rtc(ucrtc);
    Timer1_Init();
    while(1)
    {
        key_proc();
        seg_proc();
        led_relay_proc();
        get_temperature();
    }
}
```

Fence 4

十三届省赛第二套

```c
#include <STC15F2K60S2.H>
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "iic.h"
#include "ultrasound.h"
//按键
idata unsigned char key_val,key_old,key_down,key_up;
idata unsigned char key_slow_down;
//数码管
idata unsigned char seg_pos;
pdata unsigned char seg_buf[]={10,10,10,10,10,10,10,10};
idata unsigned char seg_slow_down;
idata unsigned char seg_disp_mode=0;//0-电压界面 1-参数界面 2-测距界面
//led
pdata unsigned char ucled[]={0,0,0,0,0,0,0,0};
idata bit led_flash_flag;
idata unsigned char led_flash_tick;
//ad
idata unsigned char ad_slow_down;
//距离
idata unsigned char dist_slow_down;
idata unsigned char dist_val;
idata bit dist_flag;//测距标志位 1-启动测距 0-未启动测距
//电压
idata unsigned int v_val_100x;
idata bit para_flag;//0-上限参数 1-下限参数
idata unsigned char v_para_up_10x=45;//上限参数十倍 最大值为50（5v）最小值为5（0.5v）
```

```
idata unsigned char V_para_down_10x=5;//下限参数十倍 最大值为50（5v）最小值为5（0.5v）
idata unsigned char V_para_ct_up_10x;//上限参数十倍 最大值为50（5v）最小值为5（0.5v）
idata unsigned char V_para_ct_down_10x;//下限参数十倍 最大值为50（5v）最小值为5（0.5v）
void key_proc()
{
    if(key_slow_down<10) return;
    key_slow_down=0;
    key_val=key_read();
    key_down=key_val&(key_val^key_old);
    key_up=~key_val&(key_val^key_old);
    key_old=key_val;
    if(key_down==4)
    {
        if(seg_disp_mode==0)
        {
            seg_disp_mode=2;
        }
        else if(seg_disp_mode==2)
        {
            V_para_ct_up_10x=V_para_up_10x;
            V_para_ct_down_10x=V_para_down_10x;
            para_flag=0;
            seg_disp_mode=1;
        }
        else
        {
            V_para_up_10x=V_para_ct_up_10x;
            V_para_down_10x=V_para_ct_down_10x;
            seg_disp_mode=0;
        }
    }
    if(seg_disp_mode==1)
    {
```

```
            if(key_down==5)
            {
                para_flag=!para_flag;
            }
            if(key_down==6)
            {
                if(para_flag==0)
                {
                    if(V_para_ct_up_10x>=50)
                        V_para_ct_up_10x=5;
                    else
                        V_para_ct_up_10x+=5;
                }
                else
                {
                    if(V_para_ct_down_10x>=50)
                        V_para_ct_down_10x=5;
                    else
                        V_para_ct_down_10x+=5;
                }
            }
            if(key_down==7)
            {
                if(para_flag==0)
                {
                    if(V_para_ct_up_10x<=5)
                        V_para_ct_up_10x=50;
                    else
                        V_para_ct_up_10x-=5;
                }
                else
                {
                    if(V_para_ct_down_10x<=5)
                        V_para_ct_down_10x=50;
                    else
                        V_para_ct_down_10x-=5;
```

```
            }
        }
    }
}

void seg_proc()
{
    if(seg_slow_down<80) return;
    seg_slow_down=0;
    switch(seg_disp_mode)
    {
        case 0:
            seg_buf[0]=11;
            seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=10;
            seg_buf[5]=v_val_100x/100%10+',';
            seg_buf[6]=v_val_100x/10%10;
            seg_buf[7]=v_val_100x%10;
            break;
        case 1:
            seg_buf[0]=12;
            seg_buf[1]=seg_buf[2]=10;
            seg_buf[3]=v_para_ct_up_10x/10%10+',';
            seg_buf[4]=v_para_ct_up_10x%10;
            seg_buf[5]=10;
            seg_buf[6]=v_para_ct_down_10x/10%10+',';
            seg_buf[7]=v_para_ct_down_10x%10;
            break;
        case 2:
            seg_buf[0]=13;
            seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=10;
            if(dist_flag)
            {
                seg_buf[5]=(dist_val>=100)?
dist_val/100%10:10;
                seg_buf[6]=(dist_val>=10)?
dist_val/10%10:10;
```

```
                seg_buf[7]=dist_val%10;
            }
            else
            {
                seg_buf[5]=seg_buf[6]=seg_buf[7]=14;
            }
            break;
    }
}


void ad_proc()
{
    float temp;
    if(ad_slow_down<100) return;
    ad_slow_down=0;
    ad_read(0x43);
    temp=ad_read(0x43)/51.0;
    v_val_100x=(unsigned int)(temp*100);
}

void dist_proc()
{
    if(dist_slow_down<100) return;
    dist_slow_down=0;

if(v_para_down_10x*10<v_val_100x&&v_val_100x<v_para_up_10x*
10)
    {
        dist_flag=1;
        dist_val=ut_wave_data();
    }
    else
    {
        dist_flag=0;
        dist_val=0;
```

```
    }

}
void da_proc()
{
    float temp;
    if(dist_flag==0)
    {
        da_write(0*51);
    }
    else
    {
        if(dist_val>=80) temp=5;
        else if(dist_val>20) temp=(4.0/60.0)*(dist_val-
20)+1.0;
        else temp=1;
        da_write(temp*51);
    }
}
void led_proc()
{
    switch(seg_disp_mode)
    {
        case 0://0-电压界面 1-参数界面 2-测距界面
            ucled[0]=1;
            ucled[1]=0;
            ucled[2]=0;
            break;
        case 1:
            ucled[0]=0;
            ucled[1]=0;
            ucled[2]=1;
            break;
        case 2:
            ucled[0]=0;
            ucled[1]=1;
```

```
            ucled[2]=0;
            break;
    }
    ucled[7]=led_flash_flag;

}
void Timer1_Init(void)      //1毫秒@12.000MHz
{
    AUXR &= 0xBF;           //定时器时钟12T模式
    TMOD &= 0x0F;           //设置定时器模式
    TL1 = 0x18;             //设置定时初始值
    TH1 = 0xFC;             //设置定时初始值
    TF1 = 0;                //清除TF1标志
    TR1 = 1;                //定时器1开始计时
    ET1 = 1;                //使能定时器1中断
    EA = 1;          //使能全局中断
}
void Timer1_Isr(void) interrupt 3
{
    key_slow_down++;
    seg_slow_down++;
    ad_slow_down++;
    dist_slow_down++;
    if(seg_buf[seg_pos]>20)
        seg_disp(seg_pos,seg_buf[seg_pos]-',',1);
    else
        seg_disp(seg_pos,seg_buf[seg_pos],0);
    seg_pos=(++seg_pos)%8;
    if(dist_flag)
    {
        if(++led_flash_tick==100)
        {
            led_flash_tick=0;
            led_flash_flag=!led_flash_flag;
        }
    }
```

```
    else
    {
        led_flash_tick=0;
        led_flash_flag=0;
    }
    led_disp(ucled);
}
void main()
{
    system_init();
    Timer1_Init();
    while(1)
    {
        key_proc();
        seg_proc();
        ad_proc();
        dist_proc();
        da_proc();
        led_proc();
    }
}
```

Fence 5

十四届省赛

```c
#include <STC15F2K60S2.H>
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "onewire.h"
#include "ds1302.h"
#include "iic.h"
//频率
idata unsigned int freq_time_1s;
idata unsigned int freq;
//数码管
idata unsigned char seg_pos;
pdata unsigned char seg_buf[]={10,10,10,10,10,10,10,10};
idata unsigned char seg_slow_down;
idata unsigned char seg_disp_mode;//0-时间界面 1-回显界面 2-参数界面 3-温湿度界面
idata unsigned char back_disp_mode;//回显界面子界面 0-温度回显 1-湿度回显 2-时间回显
pdata unsigned char seg_disp_mode_old[2];//0主界面 1回显界面子界面
//按键
idata unsigned char key_val,key_old,key_down,key_up;
idata unsigned char key_slow_down;
idata bit key_long_flag;//按键长按标志位
idata unsigned int key_long_tick;//按键2s长按计时器
//时间
pdata unsigned char ucrtc[]={13,3,5};

//温度
idata unsigned char T_val;//当前温度
```

```
idata unsigned char T_max;//最大温度
idata unsigned int T_average_10x;//平均温度10倍
idata unsigned int T_sum=0;//温度之和
idata unsigned char T_para=30;//温度参数 0-99
//湿度
idata unsigned char W_val;//当前湿度
idata unsigned char W_max;//最大湿度
idata unsigned int W_average_10x;//平均湿度10倍
idata unsigned int W_sum=0.0;//湿度之和
idata bit W_error_flag;//采集到的湿度数据无效标志位 1-无效 0-有效

pdata float old_TW_data[2];//前一次采集到的数据 0温度 1湿度
//触发
idata unsigned char trigger_count=0;//采集功能累计触发的次数
pdata unsigned char trigger_rtc[3];//最近一次触发数据采集功能的时间
idata bit trigger_flag;//采集功能触发标志位
idata unsigned int time_3s;//3s计时器
//光照读取
idata float  light_u_test;//光敏电阻分压采集
idata bit light_flag_old;//前一次的亮暗状态标志位 1-亮 0-暗
idata bit light_flag;//这一次的亮暗状态标志位 1-亮 0-暗
idata unsigned char ad_slow_down;
//led
pdata unsigned char ucled[]={0,0,0,0,0,0,0,0};
idata unsigned char led_flash_tick;//led闪烁0.1s计时器、
idata bit led_flash_flag;//led闪烁标志位
//按键处理函数
void key_proc()
{
    if(key_slow_down<10) return;
    key_slow_down=0;
    key_val=key_read();
    key_down=key_val&(key_val^key_old);
    key_up=~key_val&(key_val^key_old);
    key_old=key_val;
```

```
    if(seg_disp_mode!=3) //不处于温湿度界面按下按键才有效
    {
        if(key_down==4)
        {
            seg_disp_mode=(++seg_disp_mode)%3;//0-1-2之间循环
            back_disp_mode=0;//每次从时间界面进入回显界面前，把子界面置为温度回显界面
        }
        if(seg_disp_mode==1)
        {
            if(key_down==5)
                back_disp_mode=(++back_disp_mode)%3;//0-1-2之间循环
            if(back_disp_mode==2)
            {
                if(key_down==9)
                {
                    key_long_flag=1;
                }
                if(key_up==9)
                {
                    if(key_long_tick>=2000)
                    {
                        T_sum=W_sum=0;
                        T_val=W_val=0;
                        T_max=W_max=0;
                        T_average_10x=W_average_10x=0;
                        trigger_count=0;

trigger_rtc[0]=trigger_rtc[1]=trigger_rtc[2]=0;
                    }
                    key_long_flag=0;
                    key_long_tick=0;
                }
            }
        }
```

```
        if(seg_disp_mode==2)
        {
            if(key_down==8)
                if(T_para<99) T_para++;
            if(key_down==9)
                if(T_para>0) T_para--;
        }
    }
}

//信息处理函数
void seg_proc()
{
    if(seg_slow_down<80) return;
    seg_slow_down=0;
    read_rtc(ucrtc);//读取当前时间，为下面数码管显示时间做铺垫

//数码管
    switch(seg_disp_mode)
    {
        case 0: //时间
            seg_buf[0]=ucrtc[0]/10%10;
            seg_buf[1]=ucrtc[0]%10;
            seg_buf[2]=11;
            seg_buf[3]=ucrtc[1]/10%10;
            seg_buf[4]=ucrtc[1]%10;
            seg_buf[5]=11;
            seg_buf[6]=ucrtc[2]/10%10;
            seg_buf[7]=ucrtc[2]%10;
            break;
        case 1: //回显
            if(back_disp_mode==0) //温度回显
            {
                seg_buf[0]=12;
                seg_buf[1]=10;
                if(trigger_count!=0)
```

```
                {
                    seg_buf[2]=T_max/10%10;
                    seg_buf[3]=T_max%10;
                    seg_buf[4]=11;
                    seg_buf[5]=T_average_10x/100%10;
                    seg_buf[6]=T_average_10x/10%10+',';
                    seg_buf[7]=T_average_10x%10;
                }
                else
                {

seg_buf[2]=seg_buf[3]=seg_buf[4]=seg_buf[5]=seg_buf[6]=seg_
buf[7]=10;
                }
            }
            else if(back_disp_mode==1) //湿度回显
            {
                seg_buf[0]=13;
                seg_buf[1]=10;
                if(trigger_count!=0)
                {
                    seg_buf[2]=W_max/10%10;
                    seg_buf[3]=W_max%10;
                    seg_buf[4]=11;
                    seg_buf[5]=W_average_10x/100%10;
                    seg_buf[6]=W_average_10x/10%10+',';
                    seg_buf[7]=W_average_10x%10;
                }
                else
                {

seg_buf[2]=seg_buf[3]=seg_buf[4]=seg_buf[5]=seg_buf[6]=seg_
buf[7]=10;
                }
            }
            else //时间回显
```

```
            {
                seg_buf[0]=14;
                seg_buf[1]=trigger_count/10%10;
                seg_buf[2]=trigger_count%10;
                if(trigger_count!=0)
                {
                    seg_buf[3]=trigger_rtc[0]/10%10;
                    seg_buf[4]=trigger_rtc[0]%10;
                    seg_buf[5]=11;
                    seg_buf[6]=trigger_rtc[1]/10%10;
                    seg_buf[7]=trigger_rtc[1]%10;
                }
                else
                {

seg_buf[3]=seg_buf[4]=seg_buf[5]=seg_buf[6]=seg_buf[7]=10;
                }
            }
            break;
        case 2: //参数
            seg_buf[0]=15;

seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=seg_buf[5]=10;
            seg_buf[6]=T_para/10%10;
            seg_buf[7]=T_para%10;
            break;
        case 3: //温湿度
            seg_buf[0]=16;
            seg_buf[1]=seg_buf[2]=10;
            seg_buf[3]=T_val/10%10;
            seg_buf[4]=T_val%10;
            seg_buf[5]=11;
            if(W_error_flag==0)
            {
                seg_buf[6]=W_val/10%10;
                seg_buf[7]=W_val%10;
```

```
                }
                else
                {
                    seg_buf[6]=seg_buf[7]=17;
                }
                break;

        }
    }

    //其他模块处理函数
    void led_proc()
    {
        ucled[0]=(seg_disp_mode==0);
        ucled[1]=(seg_disp_mode==1);
        ucled[2]=(seg_disp_mode==3);
        if(T_val>T_para)
            ucled[3]=led_flash_flag;
        else
            ucled[3]=0;//温度实际值低于测量值，熄灭

        ucled[4]=W_error_flag;
        ucled[5]=(trigger_count>=2&&(T_val>old_TW_data[0])&&
    (W_val>old_TW_data[1]));

    }

    void get_ad()
    {
        if(ad_slow_down<100) return;
        ad_slow_down=0;
        ad_read(0x01);
        light_u_test=ad_read(0x01)/51.0;

        light_flag_old=light_flag;//先把上一次的亮暗状态赋值出去
        if(light_u_test>4.0)
```

```
            light_flag=1; //亮状态
        else
            light_flag=0; //暗状态
        //不在温湿度界面并且从亮变为暗，触发采集功能
        if(seg_disp_mode!=3&&
(light_flag_old==1&&light_flag==0))
            trigger_flag=1;
        else
            trigger_flag=0;
}

void trigger_proc()
{
    float temp;
    if(!trigger_flag) return;//如果不触发，直接返回
    old_TW_data[0]=T_val; //把上一次的温度赋值出去
    temp=rd_temperature();//读取新的温度
    T_val=(unsigned char)temp;
    if(freq>=200&&freq<=2000) //判定湿度数据是否有效 如果有效
    {
        W_error_flag=0;//湿度数据无效标志位置为0
        trigger_count++;//更新采集次数
        read_rtc(trigger_rtc);//读取触发采集的当前时间
        //更新最大温度，平均温度
        if(T_val>T_max)
            T_max=T_val;
        T_sum+=T_val;//计算温度之和
        T_average_10x=
((float)T_sum/(float)trigger_count)*10;
        //更新最新湿度数据，更新最大湿度，平均湿度
        old_TW_data[1]=W_val;
        W_val=10+(2.0/45.0)*(freq-200);;
        W_sum+=W_val;
        if(W_val>W_max)
            W_max=W_val;
```

```
            W_average_10x=
((float)W_sum/(float)trigger_count)*10;
        }
        else //如果无效
        {
            W_error_flag=1; //无效标志位置为1
        }
        //触发采集功能 储存当前显示界面，立刻跳转温湿度界面
        seg_disp_mode_old[0]=seg_disp_mode;
        seg_disp_mode_old[1]=back_disp_mode;
        seg_disp_mode=3;
        time_3s=0;            //初始化3s计时器
        trigger_flag = 0;   //  处理完清除标志

}
//定时器0的初始化函数
void Timer0_Init(void)        //1毫秒@12.000MHz
{
        AUXR &= 0x7F;            //定时器时钟12T模式
        TMOD &= 0xF0;            //设置定时器模式
        TMOD |= 0x05;            //设置定时器模式为计数器模式
        TL0 = 0x00;              //设置定时初始值
        TH0 = 0x00;              //设置定时初始值
        TF0 = 0;                 //清除TF0标志
        TR0 = 1;                 //定时器0开始计时
}

//定时器1的初始化函数
void Timer1_Init(void)        //1毫秒@12.000MHz
{
        AUXR &= 0xBF;            //定时器时钟12T模式
        TMOD &= 0x0F;            //设置定时器模式
        TL1 = 0x18;              //设置定时初始值
        TH1 = 0xFC;              //设置定时初始值
        TF1 = 0;                 //清除TF1标志
        TR1 = 1;                 //定时器1开始计时
```

```
    ET1 = 1;                    //使能定时器1中断
    EA = 1;                     //使能全局中断
}

//定时器1的服务中断函数
void Timer1_Isr(void) interrupt 3
{
    key_slow_down++;
    seg_slow_down++;
    ad_slow_down++;
    if(seg_buf[seg_pos]>20)
        seg_disp(seg_pos,seg_buf[seg_pos]-',',1);
    else
        seg_disp(seg_pos,seg_buf[seg_pos],0);
    seg_pos=(++seg_pos)%8;

    if(++freq_time_1s==1000)
    {
        freq_time_1s=0;
        freq=TH0<<8|TL0;
        TH0=TL0=0;
    }
    if(seg_disp_mode==3&&++time_3s==3000)
    {
        time_3s=0;
        seg_disp_mode=seg_disp_mode_old[0];
        back_disp_mode=seg_disp_mode_old[1];
    }

    if(key_long_flag)
    {
        if(++key_long_tick>=2000) key_long_tick=2001;
    }
    else
    {
        key_long_tick=0;
```

```
        }
        if(T_val>T_para)
        {
            if(++led_flash_tick==100)
            {
                led_flash_tick=0;
                led_flash_flag=!led_flash_flag;
            }
        }
        else
        {
            led_flash_tick=0;
            led_flash_flag=0;
        }
        led_disp(ucled);
    }
    void main()
    {
        system_init();
        set_rtc(ucrtc);
        Timer0_Init();
        Timer1_Init();
        while(1)
        {
            key_proc();
            seg_proc();
            led_proc();
            get_ad();
            trigger_proc();
        }
    }
```

十四届国赛

```c
#include <STC15F2K60S2.H>
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "onewire.h"
#include "iic.h"
#include "ultrasound.h"
//按键
idata unsigned char key_val,key_old,key_down,key_up;
idata unsigned char key_slow_down;
idata bit key_long_flag;
idata unsigned int key_long_tick;
//数码管
idata unsigned char seg_pos;
pdata unsigned char seg_buf[]={10,10,10,10,10,10,10,10};
idata unsigned char seg_slow_down;
idata unsigned char seg_disp_mode=0;//0-测距界面 1-参数界面 2工厂界面
idata bit para_disp_mode;//0-距离参数 1-温度参数
idata unsigned char gongchang_disp_mode;//0-校准值界面 1-介质设置界面 2-dac设置输出界面
//led
pdata unsigned char ucled[]={0,0,0,0,0,0,0,0};
idata bit led_flash_flag;
idata unsigned char led_flash_tick;
//温度
idata unsigned int T_slow_down;
idata unsigned int T_val_10x;
//距离
idata bit danwei_flag=0;//0-cm 1-m
```

```
idata unsigned int distance_val;
pdata unsigned int record_data[30]; // 记录下来的距离数据，每
200ms 采集一次，6 秒一共 30 次
idata unsigned int record_data_index;
idata unsigned int distance_jilu;
idata unsigned char dist_slow_down;
//参数
idata unsigned char distance_para=40;//10-90
idata unsigned char T_para=30;//0-80
idata char jiaozhun_para=0;//-90cm-90cm
idata unsigned int speed=340;//10-9990 m/s
idata unsigned char dac_xiaxian_10x=10;//1-20
idata bit jilu_flag;//"记录"标志位 0-不使能 1-使能
idata bit finish_jilu_flag;//完成记录标志位
idata unsigned int Time_6s;
idata unsigned char time200ms;
idata unsigned char da_time200ms;
idata bit da_flag;//开始进行da输出的标准位
void key_proc()
{
    if(key_slow_down<10) return;
    key_slow_down=0;
    key_val=key_read();
    key_down=key_val&(key_val^key_old);
    key_up=~key_val&(key_val^key_old);
    key_old=key_val;
    if(!jilu_flag) //如果不使能“记录”
    {
        if(key_down==4)
        {
            seg_disp_mode=(++seg_disp_mode)%3;
            led_off();//每次切换界面，关闭所有led，清空上一个界面的
led数据
            danwei_flag=0;//默认测距界面显示cm
            para_disp_mode=0;//默认进入参数界面显示距离参数
            gongchang_disp_mode=0;
```

```
            }
            if(key_down==5)
            {
                if(seg_disp_mode==0)
                {
                    danwei_flag=!danwei_flag;
                }
                else if(seg_disp_mode==1)
                {
                    para_disp_mode=!para_disp_mode;
                }
                else
                {
                    gongchang_disp_mode=
    (++gongchang_disp_mode)%3;
                }
            }
            if(key_down==8)
            {
                if(seg_disp_mode==1)
                {
                    if(para_disp_mode==0)
                    {
                        if(distance_para<90) distance_para+=10;
                    }
                    else
                    {
                        if(T_para<80) T_para++;
                    }
                }
                else if(seg_disp_mode==2)
                {
                    if(gongchang_disp_mode==0)
                    {
                        if(jiaozhun_para<90) jiaozhun_para+=5;
                    }
```

```
                else if(gongchang_disp_mode==1)
                {
                    if(speed<9990) speed+=10;
                }
                else
                {
                    if(dac_xiaxian_10x<20)
dac_xiaxian_10x++;
                }
            }
            else
            {
                jilu_flag=1;
                record_data_index = Time_6s = time200ms =
0;
                finish_jilu_flag=0;
            }
        }

        if(key_down==9)
        {
            if(seg_disp_mode==1)
            {
                if(para_disp_mode==0)
                {
                    if(distance_para>10) distance_para-=10;
                }
                else
                {
                    if(T_para>0) T_para--;
                }
            }
            else if(seg_disp_mode==2)
            {
                if(gongchang_disp_mode==0)
                {
```

```
                    if(jiaozhun_para>-90) jiaozhun_para-=5;
                }
                else if(gongchang_disp_mode==1)
                {
                    if(speed>10) speed-=10;
                }
                else
                {
                    if(dac_xiaxian_10x>1) dac_xiaxian_10x-
-;
                }
            }
            else
            {
                if(finish_jilu_flag)  //如果录入完数据了
                {
          da_flag = 1;
                    da_time200ms = 0;
          record_data_index = 0;
          finish_jilu_flag = 0;
                }
            }
        }

        if(key_long_tick>2000)
        {
            seg_disp_mode=0;
            danwei_flag=0;
            distance_para=40;
            T_para=30;
            jiaozhun_para=0;
            speed=340;
            dac_xiaxian_10x=10;
        }
        if(key_old==89)
        {
```

```
            key_long_flag=1;
        }
        else
        {
            key_long_flag=0;
            key_long_tick=0;
        }
    }
}

void seg_proc()
{
    if(seg_slow_down<80) return;
    seg_slow_down=0;
    switch(seg_disp_mode)
    {
        case 0:
            seg_buf[0]=T_val_10x/100%10;
            seg_buf[1]=T_val_10x/10%10+',';
            seg_buf[2]=T_val_10x%10;
            seg_buf[3]=11;
            if(danwei_flag==0)
            {
                seg_buf[4]=(distance_val>=1000)?
distance_val/1000%10:10;
                seg_buf[5]=(distance_val>=100)?
distance_val/100%10:10;
                seg_buf[6]=(distance_val>=10)?
distance_val/10%10:10;
                seg_buf[7]=distance_val%10;
            }
            else
            {
                seg_buf[4]=(distance_val>=1000)?
distance_val/1000%10:10;
                seg_buf[5]=distance_val/100%10+',';
```

```
                seg_buf[6]=distance_val/10%10;
                seg_buf[7]=distance_val%10;
            }
            break;
        case 1:
            seg_buf[0]=12;
            seg_buf[2]=seg_buf[3]=seg_buf[4]=seg_buf[5]=10;
            if(para_disp_mode==0)
            {
                seg_buf[1]=1;
                seg_buf[6]=distance_para/10%10;
                seg_buf[7]=distance_para%10;
            }
            else
            {
                seg_buf[1]=2;
                seg_buf[6]=T_para/10%10;
                seg_buf[7]=T_para%10;
            }
            break;
        case 2:
            seg_buf[0]=13;
            if(gongchang_disp_mode==0) //校准值设置界面
            {
                seg_buf[1]=1;
                seg_buf[2]=seg_buf[3]=seg_buf[4]=10;
                if(jiaozhun_para>=0)
                {
                    seg_buf[5]=10;
                    seg_buf[6]=(jiaozhun_para>=10)?
jiaozhun_para/10%10:10;
                    seg_buf[7]=jiaozhun_para%10;
                }
                else
                {
                    if((-jiaozhun_para)>=10)
```

```
                        {
                            seg_buf[5]=11;
                            seg_buf[6]=(-jiaozhun_para)/10%10;
                            seg_buf[7]=(-jiaozhun_para)%10;
                        }
                        else
                        {
                            seg_buf[5]=10;
                            seg_buf[6]=11;
                            seg_buf[7]=(-jiaozhun_para)%10;
                        }
                    }

                }
                else if(gongchang_disp_mode==1) //介质设置界面
                {
                    seg_buf[1]=2;
                    seg_buf[2]=seg_buf[3]=10;
                    seg_buf[4]=(speed>=1000)?speed/1000%10:10;
                    seg_buf[5]=(speed>=100)?speed/100%10:10;
                    seg_buf[6]=(speed>=10)?speed/10%10:10;
                    seg_buf[7]=speed%10;
                }
                else //dac输出界面
                {
                    seg_buf[1]=3;
                    seg_buf[2]=seg_buf[3]=seg_buf[4]=
seg_buf[5]=10;
                    seg_buf[6]=dac_xiaxian_10x/10%10+',';
                    seg_buf[7]=dac_xiaxian_10x%10;
                }
                break;
    }
}

void led_proc()
```

```
{
    unsigned char i;
    if(seg_disp_mode==0)
    {
        for(i=0;i<8;i++)
        {
            ucled[i]=(distance_val>>i)&0x01;
        }
        if(distance_val>255)
        {
            ucled[0]=ucled[1]=ucled[2]=ucled[3]=1;
            ucled[4]=ucled[5]=ucled[6]=ucled[7]=1;
        }
    }
    else if(seg_disp_mode==1)
    {
        ucled[7]=1;
        ucled[0]=ucled[1]=ucled[2]=ucled[3]=0;
        ucled[4]=ucled[5]=ucled[6]=0;
    }
    else
    {
        ucled[0]=led_flash_flag;
        ucled[1]=ucled[2]=ucled[3]=0;
        ucled[4]=ucled[5]=ucled[6]=ucled[7]=0;
    }
}

void relay_proc()
{
    if(distance_para-
5<=distance_val&&distance_para+5>=distance_val&&
(T_val_10x/10.0)<=T_para)
    {
        relay(1);
    }
```

```
    else
    {
        relay(0);
    }
}

void da_proc()
{
    idata float V_low;
    if(da_time200ms<200) return;
    da_time200ms=0;
    V_low = dac_xiaxian_10x / 10.0;    // DAC下限电压（默认
1.0V）
    if(record_data[record_data_index] >= 90)
        da_write(5*51.0);
    else if(record_data[record_data_index] <= 10)
        da_write(V_low*51.0);
    else
        da_write(((5.0 - V_low) / 80.0 *
record_data[record_data_index] + ((9*V_low-5)/8.0))*51.0);
    if(++record_data_index == 30)
        da_flag = 0;
}
void get_temperature()
{
    idata float temp;
    if(T_slow_down<300) return;
    T_slow_down=0;
    temp=rd_temperature();
    T_val_10x=(unsigned int)(temp*10);
}

void get_distance()
{
    idata unsigned int time;
    idata float temp_dist;
```

```
    if(dist_slow_down<100) return;
    dist_slow_down=0;
    time=ut_wave_data();
    temp_dist = (time*(speed/10000.0)*0.5) + jiaozhun_para;
  distance_val = (temp_dist > 0) ? (unsigned int)temp_dist
: 0; //如果temp_dist为负数，那么实际距离赋值为0
}



void Timer1_Init(void)      //1毫秒@12.000MHz
{
    AUXR &= 0xBF;           //定时器时钟12T模式
    TMOD &= 0x0F;           //设置定时器模式
    TL1 = 0x18;             //设置定时初始值
    TH1 = 0xFC;             //设置定时初始值
    TF1 = 0;                //清除TF1标志
    TR1 = 1;                //定时器1开始计时
    ET1 = 1;                //使能定时器1中断
    EA  = 1;          //使能全局中断
}

void Timer1_Isr(void) interrupt 3
{
    key_slow_down++;
    seg_slow_down++;
    T_slow_down++;
    dist_slow_down++;
    if(da_flag) da_time200ms++;
    if(seg_buf[seg_pos]>20)
        seg_disp(seg_pos,seg_buf[seg_pos]-',',1);
    else
        seg_disp(seg_pos,seg_buf[seg_pos],0);
    seg_pos=(++seg_pos)%8;
    if(key_long_flag)
    {
```

```
            if(++key_long_tick>=2000) key_long_tick=2001;
        }
        else
        {
            key_long_tick=0;
        }
        if(seg_disp_mode==2)
        {
            if(++led_flash_tick==100)
            {
                led_flash_tick=0;
                led_flash_flag=!led_flash_flag;
            }
        }
        else
        {
            led_flash_tick=0;
            led_flash_flag=0;
        }
        if(jilu_flag)
        {
            if(++time200ms == 200)
            {
          time200ms = 0;
          record_data[record_data_index] = distance_val;  // 存入数组
          record_data_index++;                            // 索引递增
            }
            if(++Time_6s==6000)
            {
                jilu_flag=0;
                Time_6s=0;
                finish_jilu_flag=1; //完成记录标志位置为1
            }
        }
```

```
    led_disp(ucled);
}
void main()
{
    system_init();
    Timer1_Init();
    while(1)
    {
        key_proc();
        seg_proc();
        led_proc();
        relay_proc();
        da_proc();
        get_temperature();
        get_distance();
    }
}
```

Fence 7

十五届省赛第一套

```c
#include <STC15F2K60S2.H>
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "ds1302.h"
#include "iic.h"
//频率
idata unsigned int Freq;//频率测量值
idata int Freq_Max=0;//最大频率值
idata unsigned int Para_Freq=2000;//频率超限参数1000-9000Hz
idata int Para_Adjust=0;//频率校准值 -900~900Hz
idata int Freq_Final;//频率数据最终结果 =频率测量值+频率校准值
idata unsigned int Time_1s;
idata bit get_freq_flag;
//按键
idata unsigned char Key_Val,Key_Old,Key_Down,Key_Up;
idata unsigned char Key_Slow_Down;

//数码管
idata unsigned char Seg_Pos=0;
pdata unsigned char Seg_Buf[]={10,10,10,10,10,10,10,10};
idata unsigned char Seg_Slow_Down;
idata unsigned char Seg_Disp_Mode;//0-频率界面 1-参数界面 2-时间界面 3-回显界面
idata bit Para_Disp_Mode;//参数界面子界面 0-超限参数界面 1-校准值参数界面
idata bit Back_Disp_Mode;//回显界面子界面 0-频率回显界面 1-时间回显界面

//led
```

```
pdata unsigned char ucled[]={0,0,0,0,0,0,0,0};
idata bit Led1_Flash_Flag;
idata bit Led2_Flash_Flag;
idata unsigned char Time1_200ms;//L1闪烁计时器
idata unsigned char Time2_200ms;//L2闪烁计时器

//时间
pdata unsigned char ucRtc[]={13,3,5};//当前时间
pdata unsigned char max_freq_time[3];//最大频率产生的时间

void Key_Proc()
{
    if(Key_Slow_Down<10) return;
    Key_Slow_Down=0;

    Key_Val=Key_Read();
    Key_Down=Key_Val&(Key_Val^Key_Old);
    Key_Up=~Key_Val&(Key_Val^Key_Old);
    Key_Old=Key_Val;

    if(Key_Down==4)
    {
        Seg_Disp_Mode=(++Seg_Disp_Mode)%4;
        Para_Disp_Mode=Back_Disp_Mode=0;//每次切换界面时，重置子
界面
    }
    if(Key_Down==5)
    {
        if(Seg_Disp_Mode==1)
            Para_Disp_Mode=!Para_Disp_Mode;
        else if(Seg_Disp_Mode==3)
            Back_Disp_Mode=!Back_Disp_Mode;
    }
    if(Seg_Disp_Mode==1)
    {
        if(Para_Disp_Mode==0)
```

```
            {
                if(Key_Down==8)
                {
                    if(Para_Freq<9000) Para_Freq+=1000;
                }
                if(Key_Down==9)
                {
                    if(Para_Freq>1000) Para_Freq-=1000;
                }
            }
            else
            {
                if(Key_Down==8)
                {
                    if(Para_Adjust<900) Para_Adjust+=100;
                }
                if(Key_Down==9)
                {
                    if(Para_Adjust>-900) Para_Adjust-=100;
                }
            }
        }
    }

    void Seg_Proc()
    {
        if(Seg_Slow_Down<80) return;
        Seg_Slow_Down=0;
        Read_Rtc(ucRtc);//读取时间
        switch(Seg_Disp_Mode)
        {
            case 0: //频率界面
                if(Freq_Final>=0)
                {
                    Seg_Buf[0]=11;
                    Seg_Buf[1]=10;
```

```
                Seg_Buf[2]=10;
                Seg_Buf[3]=(Freq_Final>=10000)?
Freq_Final/10000%10:10;
                Seg_Buf[4]=(Freq_Final>=1000)?
Freq_Final/1000%10:10;
                Seg_Buf[5]=(Freq_Final>=100)?
Freq_Final/100%10:10;
                Seg_Buf[6]=(Freq_Final>=10)?
Freq_Final/10%10:10;
                Seg_Buf[7]=Freq_Final%10;
            }
            else
            {
                Seg_Buf[0]=11;
                Seg_Buf[1]=10;
                Seg_Buf[2]=10;
                Seg_Buf[3]=10;
                Seg_Buf[4]=10;
                Seg_Buf[5]=10;
                Seg_Buf[6]=16;
                Seg_Buf[7]=16;
            }
            break;
        case 1://参数界面
            Seg_Buf[0]=12;
            Seg_Buf[2]=10;
            Seg_Buf[3]=10;
            if(Para_Disp_Mode==0) //超限参数界面
            {
                Seg_Buf[1]=1;
                Seg_Buf[4]=Para_Freq/1000%10;
                Seg_Buf[5]=Para_Freq/100%10;
                Seg_Buf[6]=Para_Freq/10%10;
                Seg_Buf[7]=Para_Freq%10;

            }
```

```
            else //校准值参数界面
            {
                Seg_Buf[1]=2;
                if(Para_Adjust>=0)  //大于等于0
                {
                    Seg_Buf[4]=10;
                    Seg_Buf[5]=(Para_Adjust>=100)?
Para_Adjust/100%10:10;
                    Seg_Buf[6]=(Para_Adjust>=10)?
Para_Adjust/10%10:10;
                    Seg_Buf[7]=Para_Adjust%10;

                }
                else //小于0
                {
                    if((-Para_Adjust)>=100)
                    {
                        Seg_Buf[4]=13;
                        Seg_Buf[5]=(-Para_Adjust)/100%10;
                        Seg_Buf[6]=(-Para_Adjust)/10%10;
                        Seg_Buf[7]=(-Para_Adjust)%10;
                    }
                    else if((-Para_Adjust)>=10)
                    {
                        Seg_Buf[4]=10;
                        Seg_Buf[5]=13;
                        Seg_Buf[6]=(-Para_Adjust)/10%10;
                        Seg_Buf[7]=(-Para_Adjust)%10;
                    }
                    else
                    {
                        Seg_Buf[4]=10;
                        Seg_Buf[5]=10;
                        Seg_Buf[6]=13;
                        Seg_Buf[7]=(-Para_Adjust)%10;
                    }
```

```
                }
            }
            break;
        case 2: //时间界面
            Seg_Buf[0]=ucRtc[0]/10%10;
            Seg_Buf[1]=ucRtc[0]%10;
            Seg_Buf[2]=13;
            Seg_Buf[3]=ucRtc[1]/10%10;
            Seg_Buf[4]=ucRtc[1]%10;
            Seg_Buf[5]=13;
            Seg_Buf[6]=ucRtc[2]/10%10;
            Seg_Buf[7]=ucRtc[2]%10;
            break;
        case 3: //回显界面
            Seg_Buf[0]=14;
            if(Back_Disp_Mode==0)//频率回显界面
            {

                Seg_Buf[1]=11;
                Seg_Buf[2]=10;
                Seg_Buf[3]=(Freq_Max>=10000)?
Freq_Max/10000%10:10;
                Seg_Buf[4]=(Freq_Max>=1000)?
Freq_Max/1000%10:10;
                Seg_Buf[5]=(Freq_Max>=100)?
Freq_Max/100%10:10;
                Seg_Buf[6]=(Freq_Max>=10)?
Freq_Max/10%10:10;
                Seg_Buf[7]=Freq_Max%10;
            }
            else //时间回显界面
            {
                Seg_Buf[1]=15;
                Seg_Buf[2]=max_freq_time[0]/10%10;
                Seg_Buf[3]=max_freq_time[0]%10;
```

```
                Seg_Buf[4]=max_freq_time[1]/10%10;
                Seg_Buf[5]=max_freq_time[1]%10;
                Seg_Buf[6]=max_freq_time[2]/10%10;
                Seg_Buf[7]=max_freq_time[2]%10;
            }
            break;
    }
}

void Led_Proc()
{
    if(Seg_Disp_Mode==0)
    {
        ucled[0]=Led1_Flash_Flag;
    }
    else
    {
        ucled[0]=0;
    }

    if(Freq_Final>Para_Freq)
    {
        ucled[1]=Led2_Flash_Flag;
    }
    else
    {
        ucled[1]=0;
    }

    if(Freq_Final<0)
    {
        ucled[1]=1;
    }

}
```

```
void AD_DA()
{
    idata float k;
    idata float b;
    if(Freq_Final<0)
    {
        Da_Write(0*51);
    }
    else if(Freq_Final<=500)
    {
        Da_Write(1*51);
    }
    else if(Freq_Final>500&&Freq_Final<Para_Freq)
    {
        k=4.0/(Para_Freq-500);
        b=((float)Para_Freq-2500.0)/(Para_Freq-500);
        Da_Write((k*Freq_Final+b)*51);
    }
    else
    {
        Da_Write(5*51);
    }

}

void Freq_Proc()
{
    if(get_freq_flag)
    {
        get_freq_flag=0;
        Freq_Final=Freq+Para_Adjust;
        if(Freq_Final >= 0 && Freq_Max < Freq_Final)//只有当
频率测量不是负数时，才更新最大值
        {
            Freq_Max=Freq_Final;
            Read_Rtc(max_freq_time);
```

```
            }
        }
    }
    //定时器0充当计数器
    void Timer0_Init(void)        //1毫秒@12.000MHz
    {
        AUXR &= 0x7F;             //定时器时钟12T模式
        TMOD &= 0xF0;             //设置定时器模式
        TMOD |= 0x05;             //设置定时器模式为计数器模式，读取P34引
    脚处的外部脉冲
        TL0 = 0x00;               //设置定时初始值
        TH0 = 0x00;               //设置定时初始值
        TF0 = 0;                  //清除TF0标志
        TR0 = 1;                  //定时器0开始计时
    }

    void Timer1_Init(void)        //1毫秒@12.000MHz
    {
        AUXR &= 0xBF;             //定时器时钟12T模式
        TMOD &= 0x0F;             //设置定时器模式
        TL1 = 0x18;               //设置定时初始值
        TH1 = 0xFC;               //设置定时初始值
        TF1 = 0;                  //清除TF1标志
        TR1 = 1;                  //定时器1开始计时
        ET1 = 1;                  //使能定时器1中断
        EA = 1;           //使能全局中断
    }

    void Timer1_Isr(void) interrupt 3
    {
        Key_Slow_Down++;
        Seg_Slow_Down++;
        if(Seg_Buf[Seg_Pos]>20)
            Seg_Disp(Seg_Pos,Seg_Buf[Seg_Pos]-',',1);
        else
            Seg_Disp(Seg_Pos,Seg_Buf[Seg_Pos],0);
```

```
        Seg_Pos=(++Seg_Pos)%8;
        //led显示
        Led_Disp(ucled);
        if(Seg_Disp_Mode==0)
        {
            if(++Time1_200ms==200)
            {
                Time1_200ms=0;
                Led1_Flash_Flag=!Led1_Flash_Flag;
            }
        }
        else
        {
            Time1_200ms=0;
            Led1_Flash_Flag=0;
        }

        if(Freq_Final>Para_Freq)
        {

            if(++Time2_200ms==200)
            {
                Time2_200ms=0;
                Led2_Flash_Flag=!Led2_Flash_Flag;
            }
        }
        else
        {
            Time2_200ms=0;
            Led2_Flash_Flag=0;
        }


        if(++Time_1s==1000)
        {
            Time_1s=0;
```

```
            get_freq_flag=1;
            Freq=(TH0<<8)|TL0;
            TH0=TL0=0;
        }
    }
    void main()
    {
        System_Init();
        Set_Rtc(ucRtc);
        Timer0_Init();
        Timer1_Init();
        while(1)
        {
            Key_Proc();
            Seg_Proc();
            Led_Proc();
            Freq_Proc();
            AD_DA();
        }
    }
```

Fence 8

十五届省赛第二套

```c
#include <STC15F2K60S2.H>
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "onewire.h"
#include "iic.h"
//按键
idata unsigned char key_val,key_old,key_down,key_up;
idata unsigned char key_slow_down;
idata unsigned char key_long_flag;
idata unsigned int key_long_tick;
//数码管
idata unsigned char seg_pos;
pdata unsigned char seg_buf[]={10,10,10,10,10,10,10,10};
idata unsigned char seg_slow_down;
idata unsigned char seg_disp_mode=0;//0-温度界面 1-dac输出控制界面 2-校准值界面
//led
pdata unsigned char ucled[]={0,0,0,0,0,0,0,0};
idata unsigned int time1_2s;
idata unsigned int time2_2s;
idata unsigned int time_3s;
idata bit led_flash_flag;
idata unsigned char led_flash_tick;
idata bit T_up_flag;//温度上升标志位
idata bit T_down_flag;//温度下降标志位
//温度
idata unsigned char T_val;
idata unsigned char T_old;
idata unsigned int T_slow_down;
```

```
idata bit baojing_flag;
//dac
idata unsigned char dac_data=100;//0-255
idata bit dac_model=0;//0-温度控制 1-手动控制
//校准值
idata char jiaozhun_para=0;//-9 - 9
idata char jiaozhun_para_ct=0;
//锁定
idata bit lock_flag=0;//1-锁定 0-解锁
void key_proc()
{
    if(key_slow_down<10) return;
    key_slow_down=0;
    key_val=key_read();
    key_down=key_val&(key_val^key_old);
    key_up=~key_val&(key_val^key_old);
    key_old=key_val;
    if(key_down==12)
    {
        switch(seg_disp_mode)
        {
            case 0:
                seg_disp_mode=1;
                break;
            case 1:
                jiaozhun_para_ct=jiaozhun_para;
                seg_disp_mode=2;
                break;
            case 2:
                seg_disp_mode=0;
                jiaozhun_para=jiaozhun_para_ct;
                break;
        }
    }
    if(key_down==16)
    {
```

```
            switch(seg_disp_mode)
            {
                case 0:
                    if(lock_flag==0)
                        relay(1);
                    break;
                case 1:
                    if(dac_data<255) dac_data+=5;
                    break;
                case 2:
                    if(jiaozhun_para_ct<9) jiaozhun_para_ct++;
                    break;
            }
        }
        if(key_down==17)
        {
            switch(seg_disp_mode)
            {
                case 0:
                    if(lock_flag==0)
                        relay(0);
                    break;
                case 1:
                    if(dac_data>0) dac_data-=5;
                    break;
                case 2:
                    if(jiaozhun_para_ct>-9) jiaozhun_para_ct--;
                    break;
            }
        }
        if(key_down==13)
        {
            key_long_flag=1;
        }
        if(key_up==13)
        {
```

```
            if(key_long_tick>=1500)
            {
                lock_flag=!lock_flag;
            }
            else
            {
                dac_model=!dac_model;
            }
            key_long_flag=0;
            key_long_tick=0;
        }
    }


    void seg_proc()
    {
        if(seg_slow_down<80) return;
        seg_slow_down=0;
        switch(seg_disp_mode)
        {
            case 0:
                seg_buf[0]=11;

    seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=seg_buf[5]=10;
                seg_buf[6]=T_val/10%10;
                seg_buf[7]=T_val%10;
                break;
            case 1:
                seg_buf[0]=12;
                seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=10;
                seg_buf[5]=(dac_data>=100)?dac_data/100%10:10;
                seg_buf[6]=(dac_data>=10)?dac_data/10%10:10;
                seg_buf[7]=dac_data%10;
                break;
            case 2:
                seg_buf[0]=13;
```

```
seg_buf[1]=seg_buf[2]=seg_buf[3]=seg_buf[4]=seg_buf[5]=10;
            if(jiaozhun_para_ct>=0)
            {
                seg_buf[6]=10;
                seg_buf[7]=jiaozhun_para_ct%10;
            }
            else
            {
                seg_buf[6]=14;
                seg_buf[7]=(-jiaozhun_para_ct)%10;
            }
            break;

    }
}

void led_proc()
{
    ucled[0]=(!dac_model);
    ucled[1]=T_up_flag;
    ucled[2]=T_down_flag;
    ucled[3]=led_flash_flag;
    ucled[7]=(!lock_flag);
}

void get_temperature()
{
    float temp_val;
    char diff;
    if(T_slow_down<300) return;
    T_slow_down=0;
    T_old=T_val;
    temp_val=rd_temperature();
    T_val=(temp_val+jiaozhun_para<0)? 0:(unsigned char)
(temp_val+jiaozhun_para);
```

```
    if(T_val>T_old)
    {
        if(T_up_flag==0)
        {
            T_up_flag=1;
            time1_2s=0;
        }
    }
    if(T_val<T_old)
    {
        if(T_down_flag==0)
        {
            T_down_flag=1;
            time2_2s=0;
        }
    }

    diff = (char)T_val - (char)T_old;
    if(diff > 1 || diff < -1)
    {
        if(baojing_flag==0)
        {
            baojing_flag=1;
            time_3s=0;
            led_flash_tick=0;
            led_flash_flag=1;
        }
    }
}

void da_proc()
{
    float temp;
    if(dac_model==0)
    {
        if(T_val>=40) temp=5;
```

```
            else if(T_val>10) temp=(1.0/10.0)*(T_val-10)+2;
            else temp=2;
            da_write((unsigned char)(temp*51));
        }
        else
        {
            da_write(dac_data);
        }
    }

    void Timer1_Init(void)        //1毫秒@12.000MHz
    {
        AUXR &= 0xBF;             //定时器时钟12T模式
        TMOD &= 0x0F;             //设置定时器模式
        TL1 = 0x18;               //设置定时初始值
        TH1 = 0xFC;               //设置定时初始值
        TF1 = 0;                  //清除TF1标志
        TR1 = 1;                  //定时器1开始计时
        ET1 = 1;                  //使能定时器1中断
        EA = 1;           //使能全局中断
    }

    void Timer1_Isr(void) interrupt 3
    {
        key_slow_down++;
        seg_slow_down++;
        T_slow_down++;

        seg_disp(seg_pos,seg_buf[seg_pos]);
        seg_pos=(++seg_pos)%8;

        if(key_long_flag)
        {
            if(++key_long_tick>=1500) key_long_tick=1501;
        }
        else
```

```
        {
            key_long_tick=0;
        }
        if(T_up_flag)
        {
            if(++time1_2s==2000)
            {
                T_up_flag=0;
                time1_2s=0;
            }
        }
        if(T_down_flag)
        {
            if(++time2_2s==2000)
            {
                T_down_flag=0;
                time2_2s=0;
            }
        }
        if(baojing_flag)
        {
            if(++led_flash_tick==200)
            {
                led_flash_tick=0;
                led_flash_flag=!led_flash_flag;
            }
            if(++time_3s==3000)
            {
                time_3s=0;
                led_flash_tick=0;
                led_flash_flag=0;
                baojing_flag=0;
            }
        }
        led_disp(ucled);
    }
```

```
void main()
{
    system_init();
    Timer1_Init();
    while(1)
    {
        key_proc();
        seg_proc();
        get_temperature();
        led_proc();
        da_proc();
    }

}
```

Fence 9

十五届国赛

```c
#include <STC15F2K60S2.H>
#include "stdio.h"
#include "string.h"
#include "led.h"
#include "init.h"
#include "seg.h"
#include "key.h"
#include "iic.h"
#include "ultrasound.h"
#include "uart.h"
#include "math.h"

#define PI 3.14

// 频率测量
idata unsigned int freq = 0;
idata unsigned int Time_1S = 0;

// 串口
idata unsigned char Uart_Rx_Flag = 0;
idata unsigned char Uart_Rx_Index = 0;
pdata unsigned char Uart_Rx_Buf[10] =
{0,0,0,0,0,0,0,0,0,0};
idata unsigned char Uart_Rx_Tick = 0;
// 数码管
idata unsigned char Seg_Pos = 0;
pdata unsigned char Seg_Buf[8] = {10,10,10,10,10,10,10,10};
idata unsigned char Seg_Disp_Mode = 0;      // 0-坐标 1-速度
2-参数
idata unsigned char seg_slow_down;
// 状态机（空闲=0，运行=1，等待=2）
```

```
idata unsigned char State_Disp_Mode = 0;

// 按键
idata unsigned char Key_Val = 0;
idata unsigned char Key_Old = 0;
idata unsigned char Key_Down = 0;
idata unsigned char Key_Up = 0;
idata unsigned char key_slow_down;
// LED
pdata unsigned char ucled[8] = {0,0,0,0,0,0,0,0};
idata bit Led_Flash_Flag = 0;
idata unsigned char Led_Flash_Tick = 0;
idata unsigned int Time_3s = 0;
idata bit Reach_Flag = 0;
idata bit L3_Flag = 0;

// AD 光敏（光敏地址0x41，夜暗标志）
idata bit Dark_Work = 0;
idata unsigned char ad_slow_down;
// 超声波
idata unsigned char Barrier_Distance = 0;
idata unsigned char us_slow_down;
// 运动参数
pdata unsigned int Destination[2] = {0,0};       // 目的坐标
pdata unsigned int Current_Location[2] = {0,0}; // 当前位置
idata unsigned int Speed_10x = 0;                // 速度x10
idata unsigned char Para_R_10x = 10;             // Rx10
(1.0)
idata char Para_B = 0;                          // B
idata unsigned char Para_R_CT_10x = 0;          // 参数设置临
时R
idata char Para_CT_B = 0;                       // 参数设置临时B
idata bit Para_Setting_Mode = 0;                // 0-R 1-B
idata bit Gotit_Flag = 0;                       // 已接收目的坐标

// 运动模型变量（线性插值方案，避免逐帧sqrt）
```

```
pdata float speed_x = 0, speed_y = 0;                // 速度分量
pdata unsigned int Start_Location[2] = {0, 0};      // 运动起始位置
pdata unsigned int Movement_Time = 0;                // 运动累计时间（0.1s单位）
idata unsigned char cnt_movement_time = 0;           // ISR内100ms计数器
idata bit Speed_Update_Flag = 0;                     // 速度更新标志

idata unsigned char ctrl_slow_down;

/* ========== 刷新速度值（用当前R/B/freq计算Speed_10x）========== */
void Refresh_Speed()
{
    float temp_speed_10x;
    temp_speed_10x = ((float)(PI * (Para_R_10x / 10.0) * freq) / 100.0f + Para_B) * 10;
    Speed_10x = (temp_speed_10x < 0) ? 0 : (unsigned int)temp_speed_10x;
}

/* ========== 重置运动状态（锚定起点+重置时间+重算速度分量）========== */
void Reset_Movement()
{
    float dx_f, dy_f, dist;

    Start_Location[0] = Current_Location[0];
    Start_Location[1] = Current_Location[1];
    Movement_Time = 0;
    cnt_movement_time = 0;

    dx_f = (float)Destination[0] - (float)Current_Location[0];
```

```
    dy_f = (float)Destination[1] -
(float)Current_Location[1];
    dist = sqrt(dx_f * dx_f + dy_f * dy_f);
    if (dist > 0)
    {
        speed_x = ((float)Speed_10x / 10.0f) * dx_f / dist;
        speed_y = ((float)Speed_10x / 10.0f) * dy_f / dist;
    }
    else
    {
        speed_x = 0;
        speed_y = 0;
    }
}

/* ========== 控制函数（线性插值，200ms调度）========== */
void Ctrl_Proc()
{
    if(ctrl_slow_down<200) return;
    ctrl_slow_down=0;
    if (State_Disp_Mode == 1)
    {
        float new_x, new_y;
        float t_sec = Movement_Time * 0.1f;

        // 障碍物保护
        if (Barrier_Distance <= 30)
        {
            State_Disp_Mode = 2;
            return;
        }

        // 线性插值计算当前位置
        new_x = (float)Start_Location[0] + speed_x * t_sec;
        new_y = (float)Start_Location[1] + speed_y * t_sec;
```

```
            // 到达检测
            if (speed_x >= 0)
            {
                if (new_x >= (float)Destination[0]) new_x =
(float)Destination[0];
            }
            else
            {
                if (new_x <= (float)Destination[0]) new_x =
(float)Destination[0];
            }
            if (speed_y >= 0)
            {
                if (new_y >= (float)Destination[1]) new_y =
(float)Destination[1];
            }
            else
            {
                if (new_y <= (float)Destination[1]) new_y =
(float)Destination[1];
            }

            Current_Location[0] = (unsigned int)(new_x + 0.5f);
            Current_Location[1] = (unsigned int)(new_y + 0.5f);

            // 两轴都到达目的地
            if (Current_Location[0] == Destination[0] &&
Current_Location[1] == Destination[1])
            {
                State_Disp_Mode = 0;
                Reach_Flag = 1;
                Time_3s = 0;
            }
        }
}
```

```
/* ========== 速度计算任务（主循环）========== */
void Speed_Calc_Proc()
{
    if (Speed_Update_Flag)
    {
            unsigned int Old_Speed_10x;
            Speed_Update_Flag = 0;

            Old_Speed_10x = Speed_10x;
            Refresh_Speed();

            // 运行态且速度实际变化时：锚定当前位置，重置时间，重算速度分量
            if (State_Disp_Mode == 1 && Speed_10x != Old_Speed_10x)
            {
                    Reset_Movement();
            }
    }
}

/* ========== 按键处理 ========== */
void Key_Proc()
{
    if(key_slow_down<10) return;
    key_slow_down=0;
    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Val ^ Key_Old);
    Key_Up = ~Key_Val & (Key_Val ^ Key_Old);
    Key_Old = Key_Val;

    // S4：启动/暂停/恢复
    if (Key_Down == 4)
    {
        if (State_Disp_Mode == 0 && Gotit_Flag == 1)    // 空闲且有目标
```

```
            {
                Gotit_Flag = 0;
                Refresh_Speed();
                Reset_Movement();
                State_Disp_Mode = 1;
            }
            else if (State_Disp_Mode == 2 && Barrier_Distance
>= 30) // 等待且障碍消失
            {
                Refresh_Speed();
                Reset_Movement();
                State_Disp_Mode = 1;
            }
            else if (State_Disp_Mode == 1)                    //
运行 → 等待
            {
                State_Disp_Mode = 2;
            }
        }

        // S5：重置当前位置（空闲态有效）
        if (Key_Down == 5)
        {
            if (State_Disp_Mode == 0)
            {
                Current_Location[0] = 0;
                Current_Location[1] = 0;
            }
        }

        // S8：界面切换
        if (Key_Down == 8)
        {
            switch (Seg_Disp_Mode)
            {
            case 0:
```

```
            Seg_Disp_Mode = 1;
            break;
        case 1:
            Para_CT_B = Para_B;
            Para_R_CT_10x = Para_R_10x;
            Seg_Disp_Mode = 2;
            Para_Setting_Mode = 0;
            break;
        case 2:
            Para_B = Para_CT_B;
            Para_R_10x = Para_R_CT_10x;
            Seg_Disp_Mode = 0;
            break;
        }
    }

    // 参数设置下的按键
    if (Seg_Disp_Mode == 2)
    {
        if (Key_Down == 9)
            Para_Setting_Mode = !Para_Setting_Mode;

        if (Key_Down == 12)
        {
            if (Para_Setting_Mode == 0)
            {
                if (Para_R_CT_10x < 20) Para_R_CT_10x++;
            }
            else
            {
                if (Para_CT_B < 90) Para_CT_B += 5;
            }
        }

        if (Key_Down == 13)
        {
```

```
            if (Para_Setting_Mode == 0)
            {
                if (Para_R_CT_10x > 10) Para_R_CT_10x--;
            }
            else
            {
                if (Para_CT_B > -90) Para_CT_B -= 5;
            }
        }
    }
}

/* ========== 数码管显示 ========== */
void Seg_Proc()
{
    if(seg_slow_down<80) return;
    seg_slow_down=0;

    switch (Seg_Disp_Mode)
    {
    case 0:  // 坐标界面
        Seg_Buf[0] = 11;
        if (State_Disp_Mode == 0)
        {
            Seg_Buf[1] = (Current_Location[0] >= 100) ? Current_Location[0] / 100 % 10 : 10;
            Seg_Buf[2] = (Current_Location[0] >= 10)  ? Current_Location[0] / 10 % 10 : 10;
            Seg_Buf[3] = Current_Location[0] % 10;
            Seg_Buf[4] = 14;
            Seg_Buf[5] = (Current_Location[1] >= 100) ? Current_Location[1] / 100 % 10 : 10;
            Seg_Buf[6] = (Current_Location[1] >= 10)  ? Current_Location[1] / 10 % 10 : 10;
            Seg_Buf[7] = Current_Location[1] % 10;
        }
```

```
        else
        {
            Seg_Buf[1] = (Destination[0] >= 100) ?
Destination[0] / 100 % 10 : 10;
            Seg_Buf[2] = (Destination[0] >= 10)  ?
Destination[0] / 10 % 10 : 10;
            Seg_Buf[3] = Destination[0] % 10;
            Seg_Buf[4] = 14;
            Seg_Buf[5] = (Destination[1] >= 100) ?
Destination[1] / 100 % 10 : 10;
            Seg_Buf[6] = (Destination[1] >= 10)  ?
Destination[1] / 10 % 10 : 10;
            Seg_Buf[7] = Destination[1] % 10;
        }
        break;

    case 1:  // 速度/障碍界面
        Seg_Buf[0] = 12;
        Seg_Buf[2] = 10;
        if (State_Disp_Mode == 1)
        {
            Seg_Buf[1] = 1;
            Seg_Buf[3] = (Speed_10x >= 10000) ? Speed_10x /
10000 % 10 : 10;
            Seg_Buf[4] = (Speed_10x >= 1000)  ? Speed_10x /
1000 % 10 : 10;
            Seg_Buf[5] = (Speed_10x >= 100)   ? Speed_10x /
100 % 10 : 10;
            Seg_Buf[6] = Speed_10x / 10 % 10 + ',';
            Seg_Buf[7] = Speed_10x % 10;
        }
        else if (State_Disp_Mode == 0)
        {
            Seg_Buf[1] = 2;
            Seg_Buf[3] = 14;
            Seg_Buf[4] = 14;
```

```
            Seg_Buf[5] = 14;
            Seg_Buf[6] = 14;
            Seg_Buf[7] = 14;
        }
        else
        {
            Seg_Buf[1] = 3;
            Seg_Buf[3] = (Barrier_Distance >= 10000) ? Barrier_Distance / 10000 % 10 : 10;
            Seg_Buf[4] = (Barrier_Distance >= 1000)  ? Barrier_Distance / 1000 % 10 : 10;
            Seg_Buf[5] = (Barrier_Distance >= 100)   ? Barrier_Distance / 100 % 10 : 10;
            Seg_Buf[6] = (Barrier_Distance >= 10)    ? Barrier_Distance / 10 % 10 : 10;
            Seg_Buf[7] = Barrier_Distance % 10;
        }
        break;

    case 2:  // 参数界面
        Seg_Buf[0] = 13;
        Seg_Buf[1] = 10;
        Seg_Buf[2] = (Para_R_CT_10x / 10 % 10) + ',';
        Seg_Buf[3] = Para_R_CT_10x % 10;
        Seg_Buf[4] = 10;

        if (Para_CT_B > 0)
        {
            Seg_Buf[5] = 10;
            Seg_Buf[6] = (Para_CT_B >= 10) ? Para_CT_B / 10 % 10 : 10;
            Seg_Buf[7] = Para_CT_B % 10;
        }
        else if (Para_CT_B < 0)
        {
            if ((-Para_CT_B) >= 10)
```

```
            {
                Seg_Buf[5] = 14;
                Seg_Buf[6] = (-Para_CT_B) / 10 % 10;
                Seg_Buf[7] = (-Para_CT_B) % 10;
            }
            else
            {
                Seg_Buf[5] = 10;
                Seg_Buf[6] = 14;
                Seg_Buf[7] = (-Para_CT_B) % 10;
            }
        }
        else
        {
            Seg_Buf[5] = 10;
            Seg_Buf[6] = 10;
            Seg_Buf[7] = 0;
        }
        break;
    }
}

/* ========== LED 与继电器 ========== */
void Led_Proc()
{
    switch (State_Disp_Mode)
    {
    case 0:
        ucled[0] = 0;
        ucled[1] = 0;
        Relay(0);
        break;
    case 1:
        ucled[0] = 1;
        ucled[1] = Dark_Work;
        Relay(1);
```

```
            break;
        case 2:
            ucled[0] = Led_Flash_Flag;
            ucled[1] = 0;
            Relay(0);
            break;
        }
        ucled[2] = L3_Flag;
}

void Get_AD()
{
        unsigned char temp_ad;
        if(ad_slow_down<100) return;
        ad_slow_down=0;
            Ad_Read(0x41);
        temp_ad = Ad_Read(0x41);
        Dark_Work = (temp_ad <= 1.2 * 51);
}

/* ========== 超声波测距 ========== */
void Get_Distance()
{
        if(us_slow_down<100) return;
        us_slow_down=0;

        Barrier_Distance = Ut_Wave_Data();
        if (Barrier_Distance == 0)
            Barrier_Distance = 255;

        if (State_Disp_Mode == 1 && Barrier_Distance <= 30)
            State_Disp_Mode = 2;
}

/* ========== 串口通信 ========== */
void Uart_Proc()
```

```
{
    unsigned int x, y;
    int result,num;
    if (Uart_Rx_Index == 0) return;
    if (Uart_Rx_Tick >= 10)
    {
        Uart_Rx_Flag = 0;
        Uart_Rx_Tick = 0;

        if (Uart_Rx_Buf[0] == '(' &&
Uart_Rx_Buf[Uart_Rx_Index - 1] == ')')
        {
            result=sscanf(Uart_Rx_Buf, "(%u,%u)%n", &x,
&y,&num);
            if (result==2&&strlen(Uart_Rx_Buf)==num)
            {
                if (State_Disp_Mode == 0)
                {
                    printf("Got it");
                    Gotit_Flag = 1;
                    Destination[0] = x;
                    Destination[1] = y;
                }
                else if (State_Disp_Mode == 2)
                {
                    printf("Wait");
                }
                else
                {
                    printf("Busy");
                }
            }
            else
            {
                printf("Error");
            }
```

```
        }
        else if (strcmp(Uart_Rx_Buf, "?") == 0)
        {
            if (State_Disp_Mode == 0)
                printf("Idle");
            else if (State_Disp_Mode == 2)
                printf("Wait");
            else
                printf("Busy");
        }
        else if (strcmp(Uart_Rx_Buf, "#") == 0)
        {
            printf("(%u,%u)", Current_Location[0],
Current_Location[1]);
        }
        else
        {
            printf("Error");
        }

        memset(Uart_Rx_Buf, 0, 10);
        Uart_Rx_Index = 0;
    }
}

/* ========== 串口中断 ========== */
void Uart1_Isr(void) interrupt 4
{
    if (RI)
    {
        Uart_Rx_Flag = 1;
        Uart_Rx_Tick = 0;
        if (Uart_Rx_Index < 10)
        {
            Uart_Rx_Buf[Uart_Rx_Index++] = SBUF;
        }
```

```
        else
        {
            Uart_Rx_Index = 0;
            Uart_Rx_Flag = 0;
            Uart_Rx_Tick = 0;
            memset(Uart_Rx_Buf, 0, 10);
        }
        RI = 0;
    }
}

/* ========== 定时器0（外部计数，测频率）========== */
void Timer0_Init(void)
{
    AUXR &= 0x7F;
    TMOD &= 0xF0;
    TMOD |= 0x05;
    TL0 = 0x00;
    TH0 = 0x00;
    TF0 = 0;
    TR0 = 1;
}

/* ========== 定时器1（1ms 系统时钟）========== */
void Timer1_Init(void)
{
    AUXR &= 0xBF;
    TMOD &= 0x0F;
    TL1 = 0x18;
    TH1 = 0xFC;
    TF1 = 0;
    TR1 = 1;
    ET1 = 1;
    EA = 1;
}
```

```
void Timer1_Isr(void) interrupt 3
{
    key_slow_down++;
    seg_slow_down++;
    ctrl_slow_down++;
    us_slow_down++;
    ad_slow_down++;
    // 数码管动态扫描
    if (Seg_Buf[Seg_Pos] > 20)
        Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos] - ',', 1);
    else
        Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos], 0);
    Seg_Pos = (++Seg_Pos) % 8;

    // 等待态 LED 闪烁 (0.1s)
    if (State_Disp_Mode == 2)
    {
        if (++Led_Flash_Tick == 100)
        {
            Led_Flash_Tick = 0;
            Led_Flash_Flag = !Led_Flash_Flag;
        }
    }
    else
    {
        Led_Flash_Tick = 0;
        Led_Flash_Flag = 0;
    }

    // 到达后 L3 亮3秒
    if (Reach_Flag)
    {
        L3_Flag = 1;
        if (++Time_3s == 3000)
        {
            Time_3s = 0;
```

```
                L3_Flag = 0;
                Reach_Flag = 0;
            }
        }

        // 运动时间累加
        if (State_Disp_Mode == 1)
        {
            if (++cnt_movement_time == 100)
            {
                cnt_movement_time = 0;
                Movement_Time++;
            }
        }

        // 每秒读频率
        if (++Time_1S == 1000)
        {
            Time_1S = 0;
            freq = (TH0 << 8) | TL0;
            TH0 = TL0 = 0;
            Speed_Update_Flag = 1;
        }

        // 串口超时计时
        if (Uart_Rx_Flag) Uart_Rx_Tick++;
        Led_Disp(ucled);
    }

    /* ========== 主函数 ========== */
    void main()
    {
        System_Init();
        Timer0_Init();
        Uart1_Init();
        Timer1_Init();
```

```
        while (1)
        {
            Ctrl_Proc();
            Speed_Calc_Proc();
            Key_Proc();
            Seg_Proc();
            Led_Proc();
            Get_AD();
            Get_Distance();
            Uart_Proc();
        }
    }
```

Fence 10

十六届省赛第一套

```c
#include <STC15F2K60S2.H>
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "onewire.h"
#include "iic.h"
#include "ultrasound.h"
#include "math.h"
idata unsigned char Key_Val,Key_Old,Key_Down,Key_Up;
idata unsigned char Key_Slow_Down;
idata bit Long_Press_Flag;//按键长按标志位
idata unsigned int Long_Press_Time_2S;//按键长按2s计时器

idata unsigned char Seg_Pos;
pdata unsigned char Seg_Buf[]={10,10,10,10,10,10,10,10};
idata unsigned char Seg_Slow_Down;
idata unsigned char Seg_Disp_Mode=0;//0-环境状态 1-运动检测 2-参数设置 3-统计数据
idata unsigned char Para_Disp_Mode;//参数设置界面的两个界面 0-温度参数 1-距离参数
idata unsigned char Temperature_Val_Para=30;//温度参数
idata unsigned char Distance_Para=30;//距离参数

idata bit Relay_Led_Lock;//继电器，led状态锁定标志位
idata bit Relay_Old;//继电器从关到开的标志位
idata unsigned int Relay_Count;//继电器吸合次数
pdata unsigned char ucled[]={0,0,0,0,0,0,0,0};
idata bit Led_Flash_Flag;//1-亮 0-灭
idata unsigned char Led_Flash_Clock;//led闪烁计时器 100ms
```

```
idata unsigned char Temperature_Val;//环境温度
idata unsigned int Temperature_Slow_Down;
idata bit Htemperature_Flag;//高温标志位 0-未得到高温 1-高温

idata unsigned char Light_Level;//光强等级
idata unsigned char AD_Slow_Down;

idata unsigned char Motion_State; //运动状态 0-静止 1-徘徊 2-跑动
idata bit Run_State;//跑动的状态
idata bit Stay_State;//静止状态
idata bit Paralysis_State;//徘徊状态
idata unsigned char Distance_Val;
idata unsigned char Distance_Old;
idata unsigned char Distance_Sub;//距离差值
idata unsigned int Distance_Slow_Down;
idata bit First_Get_Distance;//当从0->1表示已经获取里第一次的距离
idata bit State_Change_Flag;//运动状态锁定标志位 1-锁定 0-未锁定
idata unsigned int Distance_Lock_3S;//运动状态锁定3s计时器
idata bit Distance_Flag;//距离接近标志位 1-接近



void Key_Proc()
{
    if(Key_Slow_Down<10) return;
    Key_Slow_Down=0;
    Key_Val=Key_Read();
    Key_Down=Key_Val&(Key_Val^Key_Old);
    Key_Up=~Key_Val&(Key_Val^Key_Old);
    Key_Old=Key_Val;
    if(Key_Down==4)
    {
        Seg_Disp_Mode=(++Seg_Disp_Mode)%4;//0-3之间循环
        Para_Disp_Mode=0;//让参数界面默认置为温度参数子界面
```

```
        if(Seg_Disp_Mode==2)//进入参数界面，锁定
        {
            Relay_Led_Lock=1;
        }
        else//退出参数界面，取消锁定
        {
            Relay_Led_Lock=0;
        }
    }
    //在参数界面下
    if(Seg_Disp_Mode==2)
    {
        if(Key_Down==5)
        {
            Para_Disp_Mode=(++Para_Disp_Mode)%2;
        }
        if(Key_Down==8)
        {
            if(Para_Disp_Mode==0)
            {
                if(Temperature_Val_Para<80)
                    Temperature_Val_Para++;
            }
            else
            {
                if(Distance_Para<80)
                    Distance_Para+=5;
            }
        }
        if(Key_Down==9)
        {
            if(Para_Disp_Mode==0)
            {
                if(Temperature_Val_Para>20)
                    Temperature_Val_Para--;
            }
```

```
                else
                {
                    if(Distance_Para>20)
                        Distance_Para-=5;
                }
            }
        }
        //在统计界面下
        if(Seg_Disp_Mode==3)
        {
            if(Long_Press_Time_2S>=2000)
                Relay_Count=0;//清零继电器吸合次数
            if(Key_Old==89)//按键89同时按下
            {
                Long_Press_Flag=1;
            }
            else//松手或未同时按下
            {
                Long_Press_Flag=0;//标志位置0
                Long_Press_Time_2S=0;//计时器归0
            }
        }
}

void Seg_Proc()
{
    if(Seg_Slow_Down<80) return;
    Seg_Slow_Down=0;
    switch(Seg_Disp_Mode)
    {
        case 0:
            Seg_Buf[0]=11;
            Seg_Buf[1]=Temperature_Val/10%10;
            Seg_Buf[2]=Temperature_Val%10;
            Seg_Buf[3]=Seg_Buf[4]=Seg_Buf[5]=10;
            Seg_Buf[6]=12;
```

```
            Seg_Buf[7]=Light_Level;
            break;
        case 1:
            Seg_Buf[0]=13;
            Seg_Buf[1]=Motion_State+1;
            Seg_Buf[2]=Seg_Buf[3]=Seg_Buf[4]=10;
            Seg_Buf[5]=Distance_Val/100%10;//直接按正常的写就是
高位补0，如56/100=0 0%10=0
            Seg_Buf[6]=Distance_Val/10%10;
            Seg_Buf[7]=Distance_Val%10;
            break;
        case 2:
            Seg_Buf[0]=14;
            Seg_Buf[2]=Seg_Buf[3]=Seg_Buf[4]=Seg_Buf[5]=10;
            if(Para_Disp_Mode==0)
            {
                Seg_Buf[1]=11;
                Seg_Buf[6]=Temperature_Val_Para/10%10;
                Seg_Buf[7]=Temperature_Val_Para%10;
            }
            else
            {
                Seg_Buf[1]=13;
                Seg_Buf[6]=Distance_Para/10%10;
                Seg_Buf[7]=Distance_Para%10;
            }
            break;
        case 3:
            Seg_Buf[0]=12;
            Seg_Buf[1]=11;
            Seg_Buf[2]=Seg_Buf[3]=10;
            Seg_Buf[4]=(Relay_Count>=1000)?
Relay_Count/1000%10:10;
            Seg_Buf[5]=(Relay_Count>=100)?
Relay_Count/100%10:10;
```

```
            Seg_Buf[6]=(Relay_Count>=10)?
Relay_Count/10%10:10;
            Seg_Buf[7]=Relay_Count%10;
            break;
    }
}

void Led_Proc()
{
    if(!Relay_Led_Lock)//如果不锁定
    {
        if(Distance_Flag)
        {
            switch(Light_Level)
            {
                case 1:
                    ucled[0]=1;
                    ucled[1]=ucled[2]=ucled[3]=0;
                    break;
                case 2:
                    ucled[0]=ucled[1]=1;
                    ucled[2]=ucled[3]=0;
                    break;
                case 3:
                    ucled[0]=ucled[1]=ucled[2]=1;
                    ucled[3]=0;
                    break;
                case 4:
                    ucled[0]=ucled[1]=ucled[2]=ucled[3]=1;
                    break;
            }
        }
        else
        {
            ucled[0]=ucled[1]=ucled[2]=ucled[3]=0;
        }
```

```
            if(Distance_Flag==1&&Htemperature_Flag==1)
            {
                Relay(1);
                if(Relay_Old==0)
                {
                    Relay_Old=1;
                    Relay_Count++;
                }
            }
            else
            {
                Relay(0);
                Relay_Old=0;
            }
        }
        if(Stay_State)
            ucled[7]=0;
        else if(Paralysis_State)
            ucled[7]=1;
        else if(Run_State)
            ucled[7]=Led_Flash_Flag;
    }
    void Get_AD()
    {
        float temp;
        if(AD_Slow_Down<100)return;
        AD_Slow_Down=0;
        AD_Read(0x01);
        temp=AD_Read(0x01)/51.0;
        if(temp>=3.0)Light_Level=1;
        else if(temp>=2.0)Light_Level=2;
        else if(temp>=0.5)Light_Level=3;
        else Light_Level=4;
    }

    void Get_Temperature()
```

```
{
    if(Temperature_Slow_Down<300)return;
        Temperature_Slow_Down=0;
    Temperature_Val=(unsigned char)rd_temperature();;
    if(Temperature_Val>Temperature_Val_Para)
    {
        Htemperature_Flag=1;
    }
    else
    {
        Htemperature_Flag=0;
    }

}

void Get_Distance()
{
    unsigned char temp_motion_state;
    if(Distance_Slow_Down<999) return;
        Distance_Slow_Down=0;
    if(First_Get_Distance==1)
    {
        Distance_Val=Ut_Wave_Data();
        Distance_Sub=abs(Distance_Val-Distance_Old);
        Distance_Old=Distance_Val;
        Distance_Flag=(Distance_Val<Distance_Para);
        if(!State_Change_Flag)
        {
            if(Distance_Sub<5) temp_motion_state=0;
            else if(Distance_Sub<10) temp_motion_state=1;
            else temp_motion_state=2;

            //如果状态改变
            if(temp_motion_state!=Motion_State)
            {
                State_Change_Flag=1;
```

```
                Distance_Lock_3S=0;   //3s锁定计时器初始化为0
                Motion_State=temp_motion_state;
            }

        }

        if(!Relay_Led_Lock)//led和继电器不锁定才根据实际情况把下面的三个状态置0或置1
        {
            Stay_State=(Motion_State==0);
            Paralysis_State=(Motion_State==1);
            Run_State=(Motion_State==2);
        }
    }
    else
    {
        First_Get_Distance=1;//获取了第一次的距离
        Distance_Val=Ut_Wave_Data();
        Distance_Old=Distance_Val;
    }

}
void Timer1_Init(void)      //1毫秒@12.000MHz
{
    AUXR &= 0xBF;           //定时器时钟12T模式
    TMOD &= 0x0F;           //设置定时器模式
    TL1 = 0x18;             //设置定时初始值
    TH1 = 0xFC;             //设置定时初始值
    TF1 = 0;                //清除TF1标志
    TR1 = 1;                //定时器1开始计时
    ET1 = 1;                //使能定时器1中断
    EA=1;             //打开全局中断
}

void Timer1_Isr(void) interrupt 3
{
```

```
    Key_Slow_Down++;
    Seg_Slow_Down++;
    Temperature_Slow_Down++;
    Distance_Slow_Down++;
    AD_Slow_Down++;

    //数码管动态扫描
    if(Seg_Buf[Seg_Pos]>20)
        Seg_Disp(Seg_Pos,Seg_Buf[Seg_Pos]-',',1);
    else
        Seg_Disp(Seg_Pos,Seg_Buf[Seg_Pos],0);
    Seg_Pos=(++Seg_Pos)%8;


    if(Long_Press_Flag)//如果长按按键89
    {
        if(++Long_Press_Time_2S>=2000)
Long_Press_Time_2S=2001;
    }
    else//如果没有长按
    {
        Long_Press_Time_2S=0;
    }


    if(State_Change_Flag)
    {
        if(++Distance_Lock_3S==3000)
        {
            Distance_Lock_3S=0;
            State_Change_Flag=0;
        }
    }

    if(Run_State)
    {
```

```
            if(++Led_Flash_Clock==100)
            {
                Led_Flash_Clock=0;
                Led_Flash_Flag=!Led_Flash_Flag;//在0-1之间翻转
            }
        }
        else
        {
            Led_Flash_Clock=0;
            Led_Flash_Flag=0;
        }
        Led_Disp(ucled);
    }


    void main()
    {
        System_Init();
        Timer1_Init();
        while(1)
        {
            Key_Proc();
            Seg_Proc();
            Led_Proc();
            Get_AD();
            Get_Temperature();
            Get_Distance();
        }
    }
```

十六届省赛第二套

```c
#include <STC15F2K60S2.H>
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "onewire.h"
#include "iic.h"
#include "ultrasound.h"

/* ============ 按键 ============ */
idata unsigned char key_val, key_old, key_down, key_up;
idata unsigned char key_slow_down;

/* ============ 数码管 ============ */
idata unsigned char seg_pos;
pdata unsigned char seg_buf[] = {10,10,10,10,10,10,10,10};
idata unsigned char seg_slow_down;

/* ============ LED ============ */
pdata unsigned char ucled[] = {0,0,0,0,0,0,0,0};
idata unsigned char led_flash_tick;
bit led_flash_flag = 0;

/* ============ 界面模式 ============ */
idata unsigned char seg_disp_mode = 0;     // 0-测距界面 1-参数设置界面
idata unsigned char range_show_mode = 0;   // 0-初始/清除 1-单次 2-累加
idata unsigned char para_disp_mode = 0;    // 0-时间参数 1-次数参数
```

```c
/* ============ 温度与声速 ============ */
idata unsigned char T_val;
idata unsigned int T_slow_down;
idata float speed = 340;

/* ============ 测距 ============ */
idata unsigned int dist_val;        // 单次测量结果
idata unsigned int dist_sum;        // 累加测量累计结果
bit signal_flag = 0;                // 单次测量待执行
bit summation_flag = 0;             // 累加测量执行中

/* ============ 参数 ============ */
pdata unsigned char time_apart[4] = {2, 4, 6, 4};
idata unsigned char time_apart_index = 0;
pdata unsigned char dist_count[4] = {3, 5, 7, 5};
idata unsigned char dist_count_index = 0;
idata unsigned char summation_index = 0;  // 累加已完成次数

/* ============ 累加测量定时 ============ */
idata unsigned int dist_test_timer;

/* ================================================
 *  测距执行（单次/累加共用）
 *  被 key_proc 和 dist_proc 调用
 * ================================================ */
void do_distance()
{
    unsigned int temp;
    if(signal_flag)
    {
        temp = ut_wave_data(speed);
        dist_val = temp;
        signal_flag = 0;  // 单次测量完成
    }
    else if(summation_flag)
    {
```

```
            temp = ut_wave_data(speed);
            dist_sum += temp;
            summation_index++;
            if(summation_index >= dist_count[dist_count_index])
            {
                summation_flag = 0;    // 累加测量全部完成
                summation_index = 0;
            }
        }
    }

    /* ================================================
     *   按键处理
     * ================================================ */
    void key_proc()
    {
        if(key_slow_down < 10) return;
        key_slow_down = 0;

        key_val = key_read();
        key_down = key_val & (key_val ^ key_old);
        key_up = ~key_val & (key_val ^ key_old);
        key_old = key_val;

        /* --- 累加测量执行中: S8/S12 无效 --- */
        if(summation_flag == 0)
        {
            // S8: 界面切换
            if(key_down == 8)
            {
                if(seg_disp_mode == 0)
                {
                    seg_disp_mode = 1;
                    para_disp_mode = 0;
                }
                else
```

```
            {
                if(para_disp_mode == 0)
                    para_disp_mode = 1;
                else
                    seg_disp_mode = 0;
            }
        }
        // S12: 清除（仅测距界面）
        if(seg_disp_mode == 0 && key_down == 12)
        {
            dist_val = 0;
            dist_sum = 0;
            range_show_mode = 0;  // 回到初始显示
        }
    }

    /* --- 测距界面下: S4/S5 --- */
    if(seg_disp_mode == 0)
    {
        // S4: 单次测量（可立即中断累加）
        if(key_down == 4)
        {
            signal_flag = 1;
            summation_flag = 0;
            summation_index = 0;
            dist_val = 0;
            dist_sum = 0;
            range_show_mode = 1;
            do_distance();
        }
        // S5: 累加测量
        if(key_down == 5)
        {
            summation_flag = 1;
            signal_flag = 0;
            summation_index = 0;
```

```
                dist_val = 0;
                dist_sum = 0;
                dist_test_timer = 0;
                range_show_mode = 2;
                do_distance();  // 立即执行首次测距
            }
        }

        /* --- 参数设置界面: S9 --- */
        if(seg_disp_mode == 1 && key_down == 9)
        {
            if(para_disp_mode == 0)
                time_apart_index = (++time_apart_index) % 4;
            else
                dist_count_index = (++dist_count_index) % 4;
        }
    }

    /* ================================================
     *   数码管显示
     * ================================================ */
    void seg_proc()
    {
        if(seg_slow_down < 80) return;
        seg_slow_down = 0;

        switch(seg_disp_mode)
        {
            case 0: // 测距界面
                seg_buf[1] = 12; // C
                seg_buf[2] = T_val / 10 % 10;
                seg_buf[3] = T_val % 10;
                switch(range_show_mode)
                {
                    case 0: // 初始/清除 -> 熄灭
                        seg_buf[0] = 12; // C
```

```
                    seg_buf[4] = 10;
                    seg_buf[5] = 10;
                    seg_buf[6] = 10;
                    seg_buf[7] = 10;
                    break;
                case 1: // 单次 -> A
                    seg_buf[0] = 11;
                    seg_buf[4] = (dist_val >= 1000) ?
dist_val/1000%10 : 10;
                    seg_buf[5] = (dist_val >= 100)  ?
dist_val/100%10  : 10;
                    seg_buf[6] = (dist_val >= 10)   ?
dist_val/10%10   : 10;
                    seg_buf[7] = dist_val % 10;
                    break;
                case 2: // 累加 -> E
                    seg_buf[0] = 13;
                    seg_buf[4] = (dist_sum >= 1000) ?
dist_sum/1000%10 : 10;
                    seg_buf[5] = (dist_sum >= 100)  ?
dist_sum/100%10  : 10;
                    seg_buf[6] = (dist_sum >= 10)   ?
dist_sum/10%10   : 10;
                    seg_buf[7] = dist_sum % 10;
                    break;
            }
            break;

        case 1: // 参数设置界面
            seg_buf[0] = 14; // P
            seg_buf[2] = 10;
            seg_buf[3] = 10;
            seg_buf[4] = 10;
            seg_buf[5] = 10;
            seg_buf[6] = 0;  // 高位补0
            if(para_disp_mode == 0)
```

```
            {
                seg_buf[1] = 1;
                seg_buf[7] = time_apart[time_apart_index];
            }
            else
            {
                seg_buf[1] = 2;
                seg_buf[7] = dist_count[dist_count_index];
            }
            break;
    }
}

/* ================================================
 *  LED 指示
 * ================================================ */
void led_proc()
{
    // L1: 测距界面点亮
    ucled[0] = (seg_disp_mode == 0);

    // L2: 结果超范围指示
    switch(range_show_mode)
    {
        case 0: // 初始/清除
            ucled[1] = 0;
            break;
        case 1: // 单次完成后判断
            if(signal_flag == 0)
                ucled[1] = (dist_val < 20 || dist_val >
100);
            break;
        case 2: // 累加完成后判断，执行中状态不变
            if(summation_flag == 0)
                ucled[1] = (dist_sum < 20 || dist_sum >
100);
```

```
            break;
        }

        // L3: 由中断控制闪烁标志，这里直接赋值
        ucled[2] = led_flash_flag;

}

/* ================================================
 *   温度采集
 * ================================================ */
void get_temperature()
{
    float temp;
    if(T_slow_down < 300) return;
    T_slow_down = 0;
    temp = rd_temperature();
    T_val = (unsigned char)temp;
    speed = 330 + 0.6 * temp;
}

/* ================================================
 *   累加测量间隔定时触发
 * ================================================ */
void dist_proc()
{
    if(summation_flag &&
       dist_test_timer >= (unsigned
int)time_apart[time_apart_index] * 1000)
    {
        dist_test_timer = 0;
        do_distance();
    }
}

/* ================================================
```

```
 *  DAC 输出
 * ================================================ */
void da_proc()
{
    float temp = 0;
    switch(range_show_mode)
    {
        case 0: // 未完成有效测量 -> 0V
            temp = 0;
            break;
        case 1: // 单次测量完成后更新
            if(signal_flag == 0)
            {
                if(dist_val <= 20)        temp = 1;
                else if(dist_val >= 100) temp = 5;
                else                      temp =
(float)dist_val / 20.0;
            }
            break;
        case 2: // 累加测量完成后更新
            if(summation_flag == 0)
            {
                if(dist_sum <= 20)        temp = 1;
                else if(dist_sum >= 100) temp = 5;
                else                      temp =
(float)dist_sum / 20.0;
            }
            break;
    }
    da_write((unsigned char)(temp * 51));
}

/* ================================================
 *  定时器1: 1ms @ 12.000MHz
 * ================================================ */
void Timer1_Init(void)
```

```
{
    AUXR &= 0xBF;   // 定时器时钟12T模式
    TMOD &= 0x0F;
    TL1 = 0x18;
    TH1 = 0xFC;
    TF1 = 0;
    TR1 = 1;
    ET1 = 1;
    EA = 1;
}

void Timer1_Isr(void) interrupt 3
{
    key_slow_down++;
    seg_slow_down++;
    T_slow_down++;

    // 数码管扫描
    seg_disp(seg_pos, seg_buf[seg_pos]);
    seg_pos = (++seg_pos) % 8;

    // 累加测量: 计时 + L3闪烁
    if(summation_flag)
    {
        dist_test_timer++;
        if(++led_flash_tick >= 100)
        {
            led_flash_tick = 0;
            led_flash_flag = !led_flash_flag;
        }
    }
    else
    {
        dist_test_timer = 0;
        led_flash_tick = 0;
        led_flash_flag = 0;
```

```
    }

    led_disp(ucled);
}

/* ================================================
 *   主函数
 * ================================================ */
void main()
{
    system_init();
    Timer1_Init();
    while(1)
    {
        key_proc();
        seg_proc();
        led_proc();
        get_temperature();
        dist_proc();
        da_proc();
    }
}
```